

کتاب R — مقدماتی

نگارش اصلاح‌شده • کد و خروجی کنار هم • کدهای تک‌خطی بلافاصله جواب می‌دهند

آشنایی RStudio

تفاوت R – RStudio زبان برنامه‌نویسی است - RStudio یک IDE

تغییر ظاهر RStudio

برای تغییر ظاهر RStudio می‌توان از مسیر Pane Layout ← Global Options ← Tools تغییرات را اعمال کرد.

دستورات قبلی

برای دیدن دستورات قبلی می‌توان از کلید بالا استفاده نمود یا با فشردن `Ctrl +` کلید بالا فهرستی از دستورات قبلی ظاهر می‌شوند.

تکمیل دستورات

با استفاده از کلید `tab` و بعد از نوشتن حداقل ۳ حرف پیشنهاد توسط RStudio بر اساس `package` های نصب شده روی دستگاه نمایان می‌شود.

برای تغییر تنظیمات مربوط به تکمیل دستورات هم می‌توان از مسیر `Code ← Global Options ← Tools` `Completion` مراجعه کرد.

پاک کردن دستورات از Console

برای پاک کردن دستورات در Console می‌توان از کلیدهای `Ctrl+L` استفاده نمود.

استفاده از محیط Script

برای استفاده از محیط Script کافی است از منوی `File – New File – R Script` یک پنجره جدید باز کرده یا از کلید ترکیبی `Ctrl+Shift+N` استفاده نمود.

با استفاده از گوشه پایین سمت راست پنجره Script می‌توان زبان دلخواه ویرایشگر را انتخاب نمود.

برای اجرای دستورات انتخاب شده در محیط Script نویسی می‌توان از `Ctrl+Enter` و برای اجرای تمام دستورات نوشته شده `Ctrl+Shift+S` برای اجرای بدون تکرار در Console, `Ctrl+Shift+Enter` برای اجرای با تکرار در console استفاده کرد.

با کلید میانبر `Ctrl+۱` می‌توان به محیط Script و با `Ctrl+۲` به محیط Console رفت.

تغییر رنگ نوشتار

از طریق منوی Appearance ← Global Options ← Tools می‌توان رنگ و فونت نوشته‌ها را تغییر داد.
جزوه R

مقدماتی

مقدمه

یک محیط و زبان برنامه‌نویسی متن‌باز هست که برای انجام کارهای آماری و نمایش داده به‌وجود آمد.

امروز به‌جز اینها با کمک ML باهاش پیش‌بینی هم می‌کنند.

R توسط راس ایهاکا (Ross Ihaka) و رابرت جنتلمن (Robert Gentleman) در دانشگاه اوکلند نیوزلند به وجود آمد. (۱۹۹۲)

ویژگی‌ها

بسته‌های (Package) موجود فراوان

توسعه ساده package

توی مدل‌سازی آماری و نمایش بسیار قدرتمند

به راحتی با خیلی از زبان‌های امروزی ارتباط برقرار می‌کند (مانند Python، C#، C و...)

شی‌گرا است

برپایه ماتریس

بسیاری از نرم‌افزارهای آماری خودشان رو با R سازگار کردن

بسیاری از سیستم‌های هوش تجاری برخی پایگاه داده هام خودشان رو باهاش سازگار کردن.

مباحث دوره: برنامه‌نویسی مقدماتی به زبان R

۵. مبانی برنامه‌نویسی R تا ۱۰

۲. مباحث کاربردی R

۳. تابع‌نویسی

۶. ورود داده

۵. پاکسازی داده

۶. تغییر داده

۷. اکتشاف و تحلیل داده

۸. ترکیب داده

۹. تجزیه و تحلیل داده

نصب

مبانی و عملیات ریاضی

کد: عملیات پایه (r1.Rhistory)

2 + 2

[1] 4

2 - 3

[1] -1

2 * 3

[1] 6

7 / 3

[1] 2.333333

7 %/% 3

[1] 2

7 %% 3

[1] 1

2 ^ 3

[1] 8

میانبرهای (r1) RStudio

Ctrl + L -> clear console

```
# Ctrl + N -> new file
# Ctrl + Shift + N -> new R script
# Ctrl + Enter -> run selection
# Ctrl + 1 / Ctrl + 2 -> focus Source / Console
# Alt + - -> assignment <-
```

خروجی:

(no output – comments only)

R مثل ماشین حساب است

در زبان R هم مثل R می‌شود با # کامنت گذاشت
آشنایی و توضیح محیط Ctrl+L صفحه رو پاک می‌کنه.

متغیر

کد: تخصیص متغیر (r1)

```
x <- 12
```

```
[1] 12
```

```
x
```

```
[1] 12
```

```
x <- 3.14
```

```
[1] 3.14
```

```
x
```

```
[1] 3.14
```

```
x = 12
```

```
[1] 12
```

تمرین سیب/پرتقال (الگوی کد)

```
sib <- 5
```

```
[1] 5
```

```
porteghal <- 6
```

```
[1] 6
```

```
mive <- sib + porteghal
```

```
[1] 11
```

```
mive
```

```
[1] 11
```

چرا متغیر؟

- تعریف متغیر - متغیر با استفاده از - تعریف می‌شود و یک مقدار بهش تخصیص داده می‌شود
e.g. x <- 42

بازخوانی - کافیست فقط اسمش زده بشه.

تمام عملیات ریاضی روی متغیر با مقدار عددی امکان پذیره

تمرین: یک متغیر بسازین به اسم sib و یکی به اسم porteghal و مقادیر ۵ و ۶ روتوشون ذخیره کنید و بعد نمایششون بدهید بعد از نمایش با هم جمعشون کنید و توی یک متغیر به اسم میوه ذخیره کنید و حاصل جمع روشون بدهید.

انواع داده

کد: primitive types و class() (r)

```
class(12)
```

```
[1] "numeric"
```

```
class(3.14)
```

```
[1] "numeric"
```

```
class("ali")
```

```
[1] "character"
```

```
class(TRUE)
```

```
[1] "logical"
```

کد: خطاهای جمع ناسازگار و coercion (r1)

```
2 + TRUE
```

```
[1] 3
```

```
2 + FALSE
```

```
[1] 2
```

```
TRUE + FALSE
```

```
[1] 1
```

```
2 + "ali"
```

```
Error: non-numeric argument to binary operator
```

داده ها ۳ نوع هستند

انواع داده خیلی مهم هستند چون ما همه عملیات رو نمی تونیم روی تمام انواع داده انجام بدیم. (مثال از هر ۳ مدل)

توضیح که logical عملاً مقدارش ۰ یا ۱ هست.

پس ممکنه یه جاهایی توی عملیات اینجوری به مشکل بخوریم. برای فهمیدن نوع داده از تابع (class) استفاده می کنیم. (استفاده از class روی تمام مثالهای بالا)

تمرین: ۳ تا متغیر بسازین بهش از هر ۳ نوع مقدار رو بدهید و بعد با class نوعش رو نمایش بدهید.

بردارها

کد: ساخت بردار با `c()` (`r1`)

```
c(1, 2, 3, 4, 5)
```

```
[1] 1 2 3 4 5
```

```
vec1 <- c(1, 2, 3, 4, 5)
```

```
[1] 1 2 3 4 5
```

```
vec1
```

```
[1] 1 2 3 4 5
```

```
vec2 <- c("ali", "sadeghi")
```

```
[1] "ali"      "sadeghi"
```

```
vec3 <- c(TRUE, FALSE)
```

```
[1] TRUE FALSE
```

```
vec2
```

```
[1] "ali"      "sadeghi"
```

```
vec3
```

```
[1] TRUE FALSE
```

کد: coercion در بردار (`r1`)

```
c(1, 2, 3, TRUE, FALSE)
```

```
[1] 1 2 3 1 0
```

```
c(1, 2, 3, 3.14)
```

```
[1] 1.00 2.00 3.00 3.14
```

```
c(1, 2, 3, 3.14, TRUE)
```

```
[1] 1.00 2.00 3.00 3.14 1.00
```

```
c(1, 2, 3, 3.14, TRUE, "ali")
```

```
[1] "1"    "2"    "3"    "3.14" "TRUE" "ali"
```

کد: تمرین دلار/یورو (r1)

```
days <- c("Mon", "Tue", "Wed", "Thu", "Fri")
dollars <- c(140, -50, 20, -120, 240)
euros <- c(-24, -50, 100, 350, 10)
names(dollars) <- days
names(euros) <- days
dollars
euros
dollars + euros
sum(dollars)
sum(euros)
sum(dollars + euros)
sum(dollars) + sum(euros)
```

خروجی:

```
Mon Tue Wed Thu Fri
140 -50 20 -120 240
Mon Tue Wed Thu Fri
-24 -50 100 350 10
Mon Tue Wed Thu Fri
116 -100 120 230 250
[1] 230
[1] 386
[1] 616
[1] 616
[1] 616
```

کد: اندیس‌گذاری و (r1) names()

```
nums <- c(12, 13, 14, 15, 16)
```

```
[1] 12 13 14 15 16
```

```
nums
```

```
[1] 12 13 14 15 16
```

```
nums[1]
```

```
[1] 12
```

```
nums[3]
```

```
[1] 14
```

```
nums[-1]
```

```
[1] 13 14 15 16
```

```
nums[-c(1, 2)]
```

```
[1] 14 15 16
```

```
nums[-(1:3)]
```

```
[1] 15 16
```

```
info <- c("ali", "sadeghi")
```

```
[1] "ali" "sadeghi"
```

```
names(info)
```

```
names(info) <- c("firstName", "lastName")
```

```
[1] "firstName" "lastName"
```

```
info
```

```
firstName lastName  
"ali" "sadeghi"
```

```
info["firstName"]
```

```
firstName  
"ali"
```

کد: sequence و شرط روی بردار (r1)

```
(12:364)[8]
```

```
[1] 19
```

```
nums2 <- 12:364
```

```
length(nums2) = 353
```

```
nums2[9]
```

```
[1] 20
```

```
nums > 14
```

```
[1] FALSE FALSE FALSE TRUE TRUE
```

```
nums[nums > 14]
```

```
[1] 15 16
```

```
nums[nums < 13 | nums > 16]
```

```
[1] 12
```

```
nums[nums >= 13 & nums <= 16]
```

```
[1] 13 14 15 16
```

```
c(1, 2, 3) * c(10, 100)
```

```
[1] 10 200 30
```

```
length(nums2)
```

```
[1] 353
```

می‌خواهیم ببینیم چه کارایی می‌شود با بردارها کرد.

بردار چیه؟

توی R با استفاده از تابع `c()` می‌تونیم بردار بسازیم.

از بردار برای ورود و نمایش داده استفاده می‌شود (خیلی منظمی نیست داده‌ها رو دونه دونه وارد کنیم).

تمام انواع داده رو می‌شود توی بردار ذخیره کرد.

تمرین: انواع مختلف داده رو ۲ به ۲ با هم توی یک بردار ذخیره کنید. (اگر `char` داشته باشیم همه `char` میشن).

تمرین: من افتادم توی کار خرید و فروش ارز و اول هر روز دلار و یورو می‌خرم و آخر اون روز می‌فروشیم. اختلاف خرید و فروشم رو توی هر روز یادداشت می‌کنم. حالا یا سود کردم یا ضرر. فکر کنید سودبردن از دلارو یورو اینجوری باشه که توی جدول براندن می‌نویسیم. می‌خواهم ۲ تا بردار بسازیم با اسم دلار و یورو و این مقادیر و توش ذخیره کنید و نمایش بدهید.

نمایش دادیم ولی برای فهم بهتر با توجه به اینکه خروجی باید معنی دار باشه باید روزای هفته روهم ببینیم دیگه

با استفاده از تابع `names()` می‌تونیم به یک بردار نام دهیم.

تمرین: حالا با استفاده از تابع `names` روزهای هفته به ۲ بردار تخصیص بدهید.

می‌شود یک بردار ساخت و دو بار تخصیص داد.

قبل از اینکه مثال رو ادامه بدیم خوبه بدونیم اعمال ریاضی هم مثل قبل روی بردارها می‌شود انجام داد.

مثال از `char`

مثال از `num`

اعمال ریاضی روی دو بردار یا صورت نظیر به نظیر انجام می‌شود- مثال از تمام عملیات

تمرین: حالا شما یک بردار سود روزانه بسازین و بگویند توی هر روز در مجموع من سود کردم یا ضرر و مقدارش رو نشون بدهید.

برای بدست آوردن مجموع مقادیر یک بردار از تابع sum() استفاده می‌کنیم

توی مثال قبل مجموع سود دو ارز رو بدست بسازین و بعد بگویند و در کل ضرر کردم یا مربوط چقدر؟

می‌تونیم مقادیر ۲ تا عدد رو مقایسه کنیم

مقایسه حروف در lexical ، R هست (کتابخانه اش میخانه یعنی به تنظیمات دستگاه شما بستگی دارد.

می‌شود مقادیر بر روزها رو هم نظیر به نظیر-----

تمرین: حالا به من بگویند سودمون روی دلار بیشتر بوده یا یورو (با مقایسه) (کجاها)

انتخاب مقدار از توی بردار به شکل انجام می‌شود. -با استفاده از اندیس - با استفاده از اسم

در هر دو صورت از [] برای خواندن مقدار استفاده می‌شود

برای خواندن چند مقدار می‌شود از استفاده کرد. (مثال)

برای خواندن مقادیر پشت هم از می‌شود استفاده کرد (مثال)

تمرین: مقادیر شنبه و جمعه رو توی ۱ بردار به هر ۲ روش و مقادیر ۴ شنبه تا جمعه رو توی یک بردار دیگه به هر دو روش نشون بدهید.

یک کار جالب دیگه این که R می‌فهمه چه چیزهایی رو باید نمایش بده. یعنی می‌شود به‌عنوان اندیس یک شرط داد.

تمرین: یک بردار بسازین که فقط روزهایی که از دلار سود کردم و مقادیر سود اون روزها رو نشون بده.

یکی از توابع آماری مهم تابع (ncam) یا میانگین هست. که با استفاده از اون می‌تونیم میانگین مقادیر رو بدست بیاریم.

همینطور می‌شود نسبتی از مجموعه که از یک شرط پیروی میکنه رو هم بدست آورد.

ماتریس

کد: ساخت ماتریس و dim(r1)

```
matrix(data = 1:12)
```

[,1]

[1,] 1

[2,] 2

[3,] 3

[4,] 4

[5,] 5

	[6,]	6
	[7,]	7
	[8,]	8
	[9,]	9
	[10,]	10
	[11,]	11
	[12,]	12

matrix(data = 1:12, nrow = 3)				
	[,1]	[,2]	[,3]	[,4]
[1,]	1	4	7	10
[2,]	2	5	8	11
[3,]	3	6	9	12

matrix(data = 1:12, ncol = 4)				
	[,1]	[,2]	[,3]	[,4]
[1,]	1	4	7	10
[2,]	2	5	8	11
[3,]	3	6	9	12

matrix(data = 1:12, nrow = 3, ncol = 4)				
	[,1]	[,2]	[,3]	[,4]
[1,]	1	4	7	10
[2,]	2	5	8	11
[3,]	3	6	9	12

mat1 <- matrix(data = 1:12, nrow = 3, ncol = 4)				
	[,1]	[,2]	[,3]	[,4]
[1,]	1	4	7	10
[2,]	2	5	8	11
[3,]	3	6	9	12

mat1				
	[,1]	[,2]	[,3]	[,4]
[1,]	1	4	7	10
[2,]	2	5	8	11

```
[3,] 3 6 9 12
```

```
length(mat1)
```

```
[1] 12
```

```
nrow(mat1)
```

```
[1] 3
```

```
ncol(mat1)
```

```
[1] 4
```

```
dim(mat1)
```

```
[1] 3 4
```

```
matrix(data = 1:12, nrow = 2, ncol = 3, byrow = TRUE)
```

```
      [,1] [,2] [,3]  
[1,] 1 2 3  
[2,] 4 5 6
```

کد: آدرس‌دهی ماتریس (r۱)

```
mat1 <- matrix(data = 1:12, nrow = 3, ncol = 4)  
mat1[2, 2]  
mat1[1, 4]  
mat1[1, 2:4]  
mat1[1, ]  
rownames(mat1) <- c("A", "B", "C")  
colnames(mat1) <- c("a", "b", "c", "d")  
mat1  
mat1["B", "b"]  
mat1[2, "b"]  
mat1 * 10  
sum(mat1)  
colSums(mat1)
```

```
rowSums(mat1)
```

خروجی:

```
[1] 5
[1] 10
[1] 4 7 10
[1] 1 4 7 10
a b c d
A 1 4 7 10
B 2 5 8 11
C 3 6 9 12
[1] 5
[1] 5
a b c d
A 10 40 70 100
B 20 50 80 110
C 30 60 90 120
[1] 78
a b c d
6 15 24 33
A B C
22 26 30
A B C
22 26 30
```

کد: `rbind / cbind (r۱)`

```
mat2 <- matrix(data = 1:4, nrow = 2, ncol = 2)
mat3 <- matrix(data = 5:8, nrow = 2, ncol = 2)
mat2
mat3
rbind(mat2, mat3)
cbind(mat2, mat3)
result <- cbind(mat2, mat3)
result
```

خروجی:

```
[,1] [,2]
[1,] 1 3
[2,] 2 4
[,1] [,2]
[1,] 5 7
[2,] 6 8
[,1] [,2]
```

	[1,]	1	3
	[2,]	2	4
	[3,]	5	7
	[4,]	6	8
[1,]	[2,]	[3,]	[4,]
[1,]	1	3	5
[2,]	2	4	6
	[1,]	[2,]	[3,]
[1,]	1	3	5
[2,]	2	4	6
	[1,]	[2,]	[3,]
[1,]	1	3	5
[2,]	2	4	6

ماتریس چیه ؟

توی R ماتریس ها هم مثل بردارها باید مقادیرشون هم جنس باشه.

برای ساختن ماتریس از تابع () matrix استفاده می شود که ۳ تا خصیصه اصلی داره - با چه مقادیر پر بشه - سطری پر می شود یا ستونی - چند سطر یا ستون دارد

تمرین: فروش سه فیلم اول هری پاتر توی ۳ تا بردار به ما داده شده. ستون اولی هر بردار فروش Box Office امریکا و ستون دوم فروش Box Office غیر امریکاست. اول یک بردار برای Box Office بسازین و بعد با استفاده از اون یک ماتریس به اسم ماتریس سه گانه هری پاتر بسازین و اطلاعات رو توش بریزین به همیشه شکل با اسامی سطرها و ستونها کار نداشته باشین.

برای نام گذاری سطرها و ستونها ۲ تا تابع rownames () و colnames () رو داریم که به وسیله اونهای ترسیم مثل بردارها به ماتریسیون اسم بریم.

تمرین: یک بردار نام بسازید ۲ مقدار non-US,US رو بهش بدهید و تخصیص بدهید به ستونهای ماتریسون و یک بردار اسامی بسازین و اسمها رو --- توش و تخصیص بدهید به سطرها تون.

اگر لیستها رو بلد بودین می شود خیلی ساده با استفاده از list ها این کار رو با property, dimnames انجام داده. قبل خودم اینو تو ذهنتون داشته باشین تا زمان list .

اینجا هم می توانیم مثل بردارها جمع داشته باشیم. هم سطری و هم ستونی با استفاده از توابع () colSums, () rowSums,

تمرین: مجموع فروش هر فیلم رو بدست بیارین و توی یک متغیر ذخیره کرده و نمایشش بدهید.

با استفاده از توابع rbind (), cbind () می تونیم توی R چند تا بردار، ماتریس یا ترکیب اینها رو بریزیم توی یک ماتریس

تمرین: مجموع فروش سطری و ستونی رو به جدول قبل اضافه کنید و به ستونهایش اسم بدهید.

تمرین: بر گردین و به ماتریس اصلی این جدول رو هم اضافه کنید همه چیزهایی که بلدین و صرفاً جمع سطری و ستونی حساب کنید.

تمام اون روش هایی که برای خواندن یک اطلاعات از یک بردار گفتیم برای ماتریس هم برقراره با دو بُعد – سطر و ستون – اگر می‌خواهیم کل یک سطر یا ستون انتخاب می‌شود.

تمرین: کل فروش non-US رو بدست بیارین و ذخیره کنید و بعد میانگین اون رو بدست بیارین و نمایش بدهید. بعد از اون هم فقط مجموع فروش غیر US در فیلم اول و بدست بیارین.

برای ماتریس ها هم عملیات ریاضی وجود دارن با همون شرایطی که توی بردارها گفتیم و باز هم نظیر به نظیر هستن.

تمرین: فکر کنید که هر فیلم به ازای \$۵ دیده شده باشه. تعداد تماشاچی های فیلم ها رو تخمین بزنین و نمایش بدهید.

تمرین: اینجوری که خیلی غیر دقیقه ! پس بیاین این مقادیر بلیط ها رو توی یک ماتریس ذخیره کنید و تعداد دقیق بیننده ها رو بدست بیارین و نمایش بدهید.

اگر می‌خواهیم اون اتفاقاتی که توی ریاضی می‌خونیم برای ماتریس ها بیفته باید از عملگر ضرب ماتریسی %*% استفاده کنیم.

فاکتورها

کد: ordinal (r۲) و factor nominal

```
gender <- c("M", "F", "M", "F", "M", "M")
factor(x = gender)
factor(x = gender, levels = c("M", "F", "U"))
factor(x = gender, levels = c("M", "F", "U"),
      labels = c("Male", "Female", "Unisex"))
genderFac <- factor(x = gender,
                  levels = c("M", "F", "U"),
                  labels = c("Male", "Female", "Unisex"))
genderFac
```

خروجی:

```
[1] M F M F M M
Levels: F M
[1] M F M F M M
Levels: M F U
[1] Male Female Male Female Male Male
Levels: Male Female Unisex
```

```
[1] Male   Female Male   Female Male   Male
      Levels: Male Female Unisex

[1] Male   Female Male   Female Male   Male
      Levels: Male Female Unisex
```

کد: factor ترتیبی (r۲)

```
temp <- c("l", "m", "h", "vh", "vh", "h", "l")
tempFac <- factor(x = temp,
  levels = c("vl", "l", "m", "h", "vh"),
  labels = c("veryLow", "low", "mid", "high", "veryHigh"),
  ordered = TRUE)
tempFac
genderFac[1] > genderFac[2]
tempFac[1] > tempFac[3]
unclass(tempFac)
summary(tempFac)
```

خروجی:

```
[1] low      mid      high     veryHigh veryHigh high      low
      Levels: veryLow < low < mid < high < veryHigh
[1] NA
[1] FALSE
[1] 2 3 4 5 5 4 2
      attr(,"levels")
[1] "veryLow" "low"    "mid"    "high"   "veryHigh"
      veryLow    low      mid      mid      high veryHigh
           0      2      1      2      2
      veryLow    low      mid      high veryHigh
           0      2      1      2      2
```

کد: ggplot۲ و factor(cyl) (r۲)

head(mtcars)

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

```
sort(unique(mtcars$cyl))
```

```
[1] 4 6 8
```

```
library(ggplot2) # ggplot object on next lines
```

```
ggplot2 loaded OK
```

فاکتورها چین ؟ - یکی از انواع داده های آماری - برای نگهداری داده های categorical استفاده می شود (مثال)
چرا مهمه ؟ چون بعدها که می خواهیم از package ها و مدلی های آماری استفاده کنیم R باید بداند با چه نوعی از داده ها سروکار داره .
با استفاده از تابع factor می شود به R نمونه که یک مجموعه داده فاکتور هست.

Levels در واقع حالتهایی هست که برای داده ها پیش آمده .

فاکتورها دو دسته هستند -

حالا اگر بخوایم اسم لول ها رو عوض کنیم چه کار می کنیم؟ از تابع levels استفاده می کنیم.

تمرین: فکر کنید به تعداد پرسشنامه جمع کردیم و جنسیت توشون به این ترتیب بوده مرد/مرد/زن/زن/مرد
اول با حروف اختصاری بریزشون توی یک فاکتور و بعد بهشون اسم بدهید

فکر کنید که R به طور پیش فرض فاکتورها رو از کوچک به بزرگ در نظر میگیره.

تابع summary () یکی از توابع بسیار پر کاربرد و کار راه بنداز R هست. این تابع به ازای هر نوع از داده یک سری اطلاعات باز می گرداند از بردار ماتریس ractor

آدرس دهی در فاکتورها هم مثل بردارهاست و مقایسه منطقی هم همینطور فقط باید دقت بشه که مقایسه منطقی برای فاکتورهای ترتیبی - حساس تر است.

تمرین: فکر کنید که ۵ تا کارگر داریم که سرعت کارکرد نشون اینجوریه در یک فاکتور بسازید و اینهارو داخلش بریزید دقت کنید که سرعت یک فاکتور ترتیبی هست. می خواهیم بدونیم سرعت کارگر از پنجم کندتر است یا نه ؟

data.frame

کد: ساخت data.frame و (r2) EDA

```
infoDF <- data.frame(
  name   = c("ali", "hassan", "reza", "mohammad reza"),
  family = c("sadeghi aghili", "eskandari", "salehipour", "qahtarani"),
  age    = c(38, 38, 37, 36),
  major  = c("data architecture", "programming", "data engineering", "ML engineering")
)
infoDF
class(infoDF)
summary(infoDF)
str(infoDF)
```

خروجی:

```
      name      family age      major
1      ali sadeghi aghili 38 data architecture
2    hassan   eskandari 38      programming
3      reza   salehipour 37 data engineering
4 mohammad reza   qahtarani 36      ML engineering
[1] "data.frame"

      name      family      age      major
Length   : 4   Length   : 4   Min.    :36.00   Length   : 4
N.unique : 4   N.unique : 4   1st Qu.:36.75   N.unique : 4
N.blank  : 0   N.blank  : 0   Median :37.50   N.blank  : 0
Min.nchar: 3   Min.nchar: 9   Mean    :37.25   Min.nchar:11
Max.nchar:13   Max.nchar:14   3rd Qu.:38.00   Max.nchar:17
Max.      :38.00

      4 obs. of  4 variables:      'data.frame':
 $ name   : chr  "ali" "hassan" "reza" "mohammad reza"
 $ family: chr  "sadeghi aghili" "eskandari" "salehipour" "qahtarani"
 $ age    : num  38 38 37 36
 $ major  : chr  "data architecture" "programming" "data engineering" "ML engineering"
NULL
```

کد: ساخت data.frame سیارات (r2)

```
SolarSys <- data.frame(
  Planet   = c("Mercury", "Venus", "Earth", "Mars",
    "Jupiter", "Saturn", "Uranus", "Neptune"),
  Type     = rep(c("Terrestrial", "Gas"), each = 4),
  Diameter = c(0.382, 0.949, 1, 0.532, 11.209, 9.449, 4.007, 3.883),
  Rotation = c(58.64, -243.02, 1, 1.03, 0.41, 0.43, -0.72, 0.67),
  HasRing  = rep(c(FALSE, TRUE), each = 4)
```

```
)
SolarSys
dim(SolarSys)
SolarSys$Planet
SolarSys[SolarSys$Planet == "Mars", ]
```

خروجی:

```
      Planet      Type Diameter Rotation HasRing
1 Mercury Terrestrial    0.382    58.64  FALSE
2  Venus Terrestrial    0.949   -243.02  FALSE
3   Earth Terrestrial    1.000     1.00  FALSE
4    Mars Terrestrial    0.532     1.03  FALSE
5 Jupiter      Gas   11.209     0.41   TRUE
6  Saturn      Gas    9.449     0.43   TRUE
7  Uranus      Gas    4.007    -0.72   TRUE
8 Neptune      Gas    3.883     0.67   TRUE
[1] 8 5
[1] "Mercury" "Venus"  "Earth"  "Mars"   "Jupiter" "Saturn"  "Uranus"
[8] "Neptune"
      Planet      Type Diameter Rotation HasRing
4    Mars Terrestrial    0.532     1.03  FALSE
      Planet      Type Diameter Rotation HasRing
4    Mars Terrestrial    0.532     1.03  FALSE
```

کد: subset و order (r2)

```
subset(x = mtcars, cyl == 4)
subset(x = mtcars, cyl == 4 & am == 1)
a <- c(100, 10, 1000)
order(a)
a[order(a)]
sort(a, decreasing = TRUE)
```

خروجی:

```
      mpg cyl  disp  hp drat   wt  qsec vs am gear carb
Datsun 710  22.8   4 108.0  93 3.85 2.320 18.61 1  1   4    1
Merc 240D   24.4   4 146.7  62 3.69 3.190 20.00 1  0   4    2
Merc 230    22.8   4 140.8  95 3.92 3.150 22.90 1  0   4    2
Fiat 128    32.4   4  78.7  66 4.08 2.200 19.47 1  1   4    1
Honda Civic  30.4   4  75.7  52 4.93 1.615 18.52 1  1   4    2
Toyota Corolla 33.9   4  71.1  65 4.22 1.835 19.90 1  1   4    1
Toyota Corona 21.5   4 120.1  97 3.70 2.465 20.01 1  0   3    1
Fiat X1-9    27.3   4  79.0  66 4.08 1.935 18.90 1  1   4    1
Porsche 914-2 26.0   4 120.3  91 4.43 2.140 16.70 0  1   5    2
```

Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2
Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2
	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1
Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2
Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2
										[1]	2 1 3
										[1]	10 100 1000
										[1]	1000 100 10
										[1]	1000 100 10

مرور انواع داده – در تمام انواع تا الان فقط مجاز به استفاده از یک نوع داده بودیم. در حالی که در واقعیت با انواع مختلفی از داده در پرسشنامه مواجهیم – data.frame

در data.frame سطرها مشاهدات و ستونها متغیرهای ما هستند.

Land کردن mtcars

به عنوان کسی که داره با داده کاری کنه خیلی مهمه که نگاه اجمالی به داده هایی که باهاشون سروکار داریم بنذاریم.

از تابع head(), tail() برای اینکار استفاده می شود به طور پیش فرض این ۲ تابع ۶ سطر بازگردانند.

یک راه دیگه شناختن مربع داده ها استفاده از تابع str() است. که به ما تعداد مشاهدات، تعداد متغیرها، نوع داده، نوع داده متغیرها و..... رومی گوید .

() data.frame بسازیم و روی اون پیش بریم . با استفاده از dataset حالا بریم یک

تمرین: می خواهیم یک data.frame ایجاد کنیم و اطلاعاتی رو راجع به سیارات داخلش بریزیم اول ۵ بردار بسازین و داده ها رو توش وارد کنید و بعد از روش data.frame بسازین.

تمرین: حالا ساختار داده اش رو با استفاده از Str() ببین توی data.frame هم آدرس دهی مثل ماتریس هست.

تمرین: مشخصات عطارد و اطلاعات مریخ رو نشون بدهید و یادتون هست که می شود در ماتریس ها با استفاده از اسامی مقادیر رو خونه. توی data.frame هم به همین ترتیب هست. مثلا

می شود هم به ازای هر متغیر از \$ استفاده کرد.

تمرین: با این روش یک بردار از حلقه با سیارات بسازین و بعد با استفاده از اون مشخصات تمام سیاره هایی که حلقه دارن رو نشون بدهید.

همه کارهای مثال قبل رو می‌شود باید تمامی به اسم subset () انجام داد.

۲ Subset تا Arguman داره. اولی انتخاب میکنه دیتا رو. دومی شرط و سومی که با select میاد می‌تونین بگویی. کدام ستونها رو می‌خواین انتخاب کنید.

تمرین: subset اطلاعات سیاراتی که قطرشون از ۱ کوچکتر هست رو نشون بدهید.

خیلی وقتها پیش میاد که ما می‌خواهیم داده‌ها رو مرتب کنیم. (مثال) مرتب کردن محصولات بر حسب فروش، نمره‌ها در یک کلاس، محبوبیت یک فیلم و ... با استفاده از تابع order () می‌تونیم داده‌ها رو مرتب کنیم.

تمرین: حالا شما از Df مربوط به سیارات، کل سیارات رو بر اساس قطر مرتب کنید و همه اطلاعاتشون رو نشون بدهید.

List

کد: list و [] / \$ (r۲)

```
lst <- list(vec = dollars, fac = genderFac, mat = mat1, df = SolarSys)
lst$fac
lst$fac[3]
lst$df$Planet
names(lst)
```

خروجی:

```
[1] M F M F M M
Levels: F M
[1] M
Levels: F M
[1] "Mercury" "Mars"
[1] "vec" "fac" "mat" "df"
[1] "vec" "fac" "mat" "df"
```

لیست‌ها هم یک نوع از داده‌ها هستند که نه برایشون اندازه مهمه و نه جنس داده. (توی data.frame های ستونها می‌تونستن با هم فرق کنن ولی نوع داده در هر ستون یکسان است) پس توی list می‌تواند تمام تمام فرمت های داده ای که تا الان یاد گرفتیم بعلاوه خود list می‌تونه قرار بگیره.

برای ساخت لیست از تابع list () استفاده می‌کنیم.

در list به هر component می‌توان یک اسم اختصاصی داد. مثلا (list(name۱,comp۱,...) یا می‌شود مثل قبل از تابع names () استفاده کرد.

تمرین: یک لیست با این اطلاعاتی که می‌گم بسازین

اسم فیلم . the shining

بازیگران.

توی لیست برای دسترسی به هر component از `[[ ]]` یا اگر اسم داره از `$` استفاده می‌کنیم و برای رسیدن به جزییات قدی component مثل قبل عمل می‌کنیم.

تمرین: با آدرس دهی، اسامی بازیگران رو نشون بدهید و بعد از توش بازیگر دوم رو نشون بدهید. کافیه با استفاده از `[[ ]]` یک اندیس جدید ایجاد کنیم و به اون مقدار بریم. داخل اون می‌تواند اسم یا اندیس شی جدید با شما

تمرین: سال ۱۹۸۰ رو با اسم به list قبل اضافه کنید و بعد ساختار list رو نمایش بدهید.

مباحث کاربردی R

کاربرد عملگرهای `! , | , &`

بردارهای منطقی TRUE/FALSE (در عملگرهای دوتایی recycling انجام می‌شود)

تمرین: داده‌های `log.df` رو import کنید. اینها داده‌های `view` مربوط به کارکنان هستند و برای ما مهمه که بدونیم کدوم کارکنان چقدر دیده میشن حالا می‌خواهم که شما کل داده‌های مربوط به روز دوم رو انتخاب کنید، و بعد بین چند نفر بردن که بیشتر از ۲۵ بازدید یا کمتر از ۵ بازدید داشت.

If (شرط)

کد: if ساده (Rhistry.۳)

```
x <- 2
if (x %% 2 == 0) {
  print("even")
} else {
  print("odd")
}
```

خروجی:

```
[1] "even"
[1] "odd"
[1] "odd"
```

کد: if / else if / else (۳)

```
x <- 1
if (x %% 2 == 0) {
  print("divisible by 2")
} else if (x %% 3 == 0) {
  print("divisible by 3")
} else {
  print("neither divisible by 2 nor 3")
}
```

خروجی:

```
[1] "neither divisible by 2 nor 3"
[1] "neither divisible by 2 nor 3"
```

کد: تمرین آپارات/YouTube (الگو)

```
aparat <- 5
youtube <- 9
if (aparat >= 15 & youtube >= 15) {
  print((aparat + youtube) * 2)
} else if (aparat < 10 | youtube < 10) {
  print((aparat + youtube) * 0.05)
} else {
  print(aparat + youtube)
}
```

خروجی:

```
[1] 0.7
[1] 0.7
```

شرطها (در شرطها یک مقایسه منطقی انجام می‌شود) یک مقایسه منطقی انجام میدن. در صورتی که نتیجه اش TRUE باشه یک سری کارهای کنن و اگر نباشه یکسری کارهای دیگه.

توضیح elseif

دقت کنید که اگر شروطتون هم پوشانی داشته باشن مابقی شروط کل چک نش

تمرین: ۲ تا متغیر تعریف کنید توی (مثلا media) یکی مقدار Aparat رو بریزین و توی اون یکی رو بهش مقدار بازدیدها یعنی عدد ۱۴ رو برین. چک کنید اگر متغیر اول برابر با Aparat بود پیغامی نشون بده که این اطلاعات

مربوط به Aparat هست و اگر نبود بنویسه ناشناخته. همینطور چک کنید اگر بازدیدها بیشتر از ۱۵ بود بنویسه پر بازدید و اگر نبود بنویسه بیشتر تلاش کنید. (۲ تا شرط)

تمرین: ۲ تا متغیر آپارات و YouTube بسازین. آپارات رو بهش مقداره و YouTube رو ۹ بدهید. حالا یک سری شروط بنویسن به این صورت

-اگر هر دوی آپارات و یوتیوب ۱۵ یا بیشتر بازدید داشتن تعداد مجموع بازدیدها رو ضرب در ۲ کنید و نشون بدهید (به عنوان امتیاز) .

- اگر هر دوی آپارات و یوتیوب کمتر از ۱۰ یا بیشتر بازدید داشتن تعداد مجموع بازدیدها رو ضرب در ۰.۵ کنید و نشون بدهید (به عنوان امتیاز) .

-در غیر این صورت فقط مجموع بازدیدها رو نشون بدهید.

While loop

کد: while و (۳) state changer

```
x <- 10
while (x > 0) {
  print(x)
  x <- x - 1
}
```

خروجی:

```
[1] 10
[1] 9
[1] 8
[1] 7
[1] 6
[1] 5
[1] 4
[1] 3
[1] 2
[1] 1
```

کد: while با break (الگوی تمرین)

```
speed <- 48
while (speed > 30) {
  if (speed > 48) {
    speed <- speed - 11
    print("slow down a lot")
  } else {
    speed <- speed - 6
    print("slow down")
  }
}
```

```
}  
speed
```

خروجی:

```
[1] "slow down"  
[1] "slow down"  
[1] "slow down"  
[1] 30  
[1] 30
```

توضیح while (تا وقتی اجرا می‌شود که شرط برقرار هست)

تمرین: مثال قبل رو با این تغییرات حل کنید. بازم می‌خواهیم تا وقتی سرعت بزرگتر از ۳۰ هست سرعت رو کم کنیم. اگر بیشتر از ۴۸ بود از اون ۱۱ تا کم کنیم و پیغام بدیم که سرعت رو خیلی کم کن. اگر نه هم ۶ تا کم کنیم و بگیم سرعت رو کم کن.

Break در صورتی استفاده می‌شود که بخوایم بعد از اتفاق افتادن یک شرایطی دیگه حلقه رو ادامه ندیم. مثلاً بعد از پیدا کردن یک مقدار خاص توی یک بر دار باقی اسامی خوانده نشوند.

تابع paste () از طریق این تابع می‌تونیم مقادیر مختلف رو به هم بچسبونیم. این تابع مقادیر رو به char تبدیل می‌کنه و به هم می‌چسبونه.

تمرین: یک متغیر با مقدار اولیه ۱ بسازین و تا وقتی که مقدارش کوچکتر از ۱۰ بود مقدارش رو ضرب در ۳ کنید و نمایش بدهید. اگر ۳ تا بر ۸ قابل تقسیم بود ۳ تا رو نمایش بدهید و از حلقه خارج بشین می‌خواهیم در هر بار اجراهم ۱ واحد به اضافه کنیم.

For loop

کد: for روی بردار (۳)

```
names <- c("ali", "samin", "arshia", "hooman", "arman", "chamin")  
for (name in names) {  
  print(name)  
}
```

خروجی:

```
[1] "ali"  
[1] "samin"
```

```
[1] "arshia"  
[1] "hooman"  
[1] "arman"  
[1] "chamin"  
--- break ---  
[1] "ali"  
[1] "samin"  
[1] "arshia"  
[1] "found!"  
--- next ---  
[1] "ali"  
[1] "samin"  
[1] "arshia"  
[1] "arman"  
[1] "chamin"
```

کد: paste و for روی (۳) seq

```
paste("ali", "sadeghi")
```

```
[1] "ali sadeghi"
```

```
paste("ali", "sadeghi", sep = "")
```

```
[1] "alisadeghi"
```

```
paste("ali", "sadeghi", sep = "-")
```

```
[1] "ali-sadeghi"
```

```
paste("x", 1:6, sep = "")
```

```
[1] "x1" "x2" "x3" "x4" "x5" "x6"
```

```
paste(c("ali", "hassan"), c("sadeghi", "eskandari"))
```

```
[1] "ali sadeghi" "hassan eskandari"
```

از حلقه for زمانی استفاده می‌کنیم که بخوایم یک اتفاقی به تعداد مشخصی بیفته. مثلاً

بر خلاف خیلی از زبونها مستقیما حرکت می‌کند روی خود مجموعه.
بدونین که یکجور دیگه هم می‌شود forloop رو نوشت.
تابع length () به شما طوری یک آرایه رو می‌گوید. (اندازه اش). روی بردارها یا فاکتورها هم کار میکنه.

تمرین: لیست از این اطلاعات بسازین و با حلقه اطلاعات رو بخونین.

نام شهر = New York

جمعیت = ۵۸۳۷-۸۴

شهرک ها = منهتن و برونکس و بروکلین و کونیز و جزیره استاتن پایتخت نیست

تمرین: یک ماتریس ۳ و ۳ بسازین و تک تک عناصرش رو با استفاده از for نمایش بدهید جوری که بنویسه عنصر مربوط به سطر فلان و ستون فلان انقدره. و یک عبارت دیگه هم توی حلقه ها داریم به اسم next که کارش اینه که از روی یک قسمتی از حلقه بپره.

یک تابع strsplit که ۲ تا آرگومان اصلی داره یکی یک متن و دیگری character که می‌خواد با استفاده از اون متن رو بشکونه

تمرین: یک متغیر تعریف کنید و توش seyed Ali sadeghi Aghili رو ذخیره کنید. با استفاده از strsplit و حرف به حرف بشکنیدش و توی یک متغیر ذخیره اش کنید. ساختارش رو با استفاده از تابع str بینین و بعد یک متغیر دیگه بسازین که تعداد ها رو بشماره و مقدار اولیه اش رو بزارین دقت کنید که به ازای هر باید یک واحد به شمارنده اضافه بشه و اگر به g رسید از حلقه خارج بشه و نتیجه رو چاپ کنه.

Function

کد: تعریف تابع (۲-۳.Rhistory)

```
triple <- function(num) {  
  return(num * 3)  
}  
triple(num = 12)
```

```
triple(num = 10)
result <- triple(num = 12)
result
```

خروجی:

```
[1] 36
[1] 30
[1] 36
[1] 36
```

کد: آرگومان و default (۳-۲)

```
multiply <- function(num1, num2) {
  return(num1 * num2)
}
multiply(num1 = 12, num2 = 4)
multiply <- function(num1 = 1, num2) {
  return(num1 * num2)
}
multiply(num2 = 4)
sayHello <- function() { return("Hello") }
sayHello()
num2pow <- function(num, pow = 1) {
  return(num ^ pow)
}
num2pow(12)
num2pow(12, 2)
```

خروجی:

```
[1] 48
[1] 4
[1] "Hello"
[1] "2026-09-20"
[1] 12
[1] 144
[1] 144
```

کد: توابع پایه و sample (۳-۲)

```
mean(1:5)
```

```
[1] 3
```

```
sd(1:5)
```

```
[1] 1.581139
```

```
abs(c(-1, 2, 3, -4, 5))
```

```
[1] 1 2 3 4 5
```

```
seq(from = 1, to = 20, by = 2)
```

```
[1] 1 3 5 7 9 11 13 15 17 19
```

```
seq(from = 20, to = 1, by = -3)
```

```
[1] 20 17 14 11 8 5 2
```

```
rev(1:10)
```

```
[1] 10 9 8 7 6 5 4 3 2 1
```

```
sample(x = 1:10, size = 1)
```

```
[1] 9
```

```
sample(x = 1:10, size = 3, replace = TRUE)
```

```
[1] 9 4 7
```

كد: تمرين تابع تاس (الگو)

```
dice <- function(verbose = FALSE) {  
  a <- sample(x = 1:6, size = 1)  
  b <- sample(x = 1:6, size = 1)  
  if (verbose) print(paste("dice1 =", a, ", dice2 =", b))  
  return(a + b)  
}  
dice(verbose = TRUE)
```

خروجی:

```
[1] "dice1 = 1 , dice2 = 5"
[1] 6
[1] 6
```

تعریف تابع (جعبه سیاه)

تابع args () در مقابل help

تابع mean ()

تابع abs ()

تابع sum()

ایجاد function

هدف مفهوم black box می‌خواهیم ۲ تابع بسازیم که مقادیر رو ۳ برابر کنه.

Return رو می‌شود گذاشت. می‌شود نگذاشت چون آخرین خط function ها همیشه return هست.

می‌تونیم تابع با هر چند تا Arg که می‌خواهیم بنویسیم.

تعریف e.g – default قبل و g=۰

تمرین: یه تابع بنویسید که ۳ تا عدد بگیره، قدر مطلقشان رو با هم جمع کنه و نتیجه رو نمایش بده.

تابع بدون Arg – تاسی که از بیرون چیزی نمی‌خواد. خودش یکسری چیزها رو حساب میکنه و نتیجه رو باز می‌گرداند. مثلاً می‌خواد زمان

تابع بدون Arg – سیستم رو بگیره نمایش بده یا – eg

تابع بدون Arg – عدد تصادفی تولید میکنه

تمرین: یه تابع بنویسید که بدون Arg باشه، ۲ تا عدد از ۱ تا ۶ تولید کنه و با هم جمع کنه و بعد نتیجه رو نمایش بده.

تمرین: تابع رو تغییر بدهید که یک Arg بگیره یا پیشفرض false . اگر مقدار tevs بهش داده شد بگه که هر عدد هر تاس چقدر بوده.

تمرین: یعنی دو تا بردار داریم از بازدیدهای YouTube aparat می‌خواهیم به تابع بنویسن که اگر متد و بازدیدها بیشتر از ۱۵ بود به پیغام بده که video محبو--- و مقدار رو برگردونه و در غیر این صورت چاپ کنه نامحبوب و ۵ رو برگردونه بعد به تابع دیگه بنویسید که مقدار نتیجه تابع قبل و به مقدار False, Teus بگیره. اگر Trvs بود مجموع مقادیر بیشتر از ۱۵ روهم نمایش بده و در غیر این صورت نده.

Package

کد: (۲-۲) library / detach / search

<code>.libPaths()</code>
<code>[1] "C:/Users/alisa/AppData/Local/R/win-library/4.6"</code>

<code>search()</code>
[1] ".GlobalEnv" "package:stats" "package:graphics" [4] "package:grDevices" "package:utils" "package:datasets" [7] "package:methods" "Autoloads" "package:base"

<code>head(installed.packages()[, "Package"], 8)</code>
A3 abc abc.data abcrf abind acepack
"A3" "abc" "abc.data" "abcrf" "abind" "acepack"
AICcmmodavg airports
"AICcmmodavg" "airports"

Package چیه ؟ (همه توابعی که ما به صورت built-in استفاده می‌کنیم عنصری از Package ها متن) .

بعضی از Package ها به صورت پیش‌فرض نصب می‌شوند به همراه نصب محیط و base هستند.

پکیج‌ها معمولاً از طریق CRAN نصب می‌شن. ولی برای اینکه بتونیم از Package ها استفاده کنیم باید آن‌ها را load کنیم که برای اینکار از تابع

اضافه می‌شن. برای R می‌شوند در واقع به یک لیست قابل جستجویی برای load ها Package وقتی استفاده کنید `search()` رو بتونیم ببینیم. می‌تونین از تابع `list` اینکه این این تابع به فرق . `require()` ها داریم به نام Package کردن `load` یک تابع دیگه هم برای استفاده `library` نصب شده باشه و از Package داره اونهم اینکه اگر یک `library()` بزرگ با فقط `require` بر می‌خوریم که باعث توقف برنامه می‌شود ولی با استفاده از `error` کنیم به یک می‌گیرم `warning` یک

خانواده apply

خیلی وقتها ما می‌خواهیم که یک کار تکراری رو روی یک مجموعه از داده هامون انجام بدیم. اینجاست که خانواده apply خیلی مفید هستن.

lapply

تابع eg – () lapply می‌خواهیم ببینیم هر شهر چند حرفه

می‌شود func هایی که خودمان نوشته‌ایم هم استفاده کنیم من می‌تونم زباله هام رو کیلویی ۳ هزار تومن بفروشم.

lapply همیشه list باز می‌گرداند . برای خارج کردنش از فرمت list می‌شود از تابع unlist() استفاده کرد. می‌شود از تابع lapply برای func های با چند آرگومان هم استفاده کرد. مثلاً تابع قبل رو می‌خواهیم هر دفعه بهش بگیریم ضربدر چه عددی بکنه.

تابع tolower کافیه مقدار باقی آرگومانها رو بلافاصله بعد از تابع ذکر کنیم.

تمرین: یک بردار از اسامی آدمها و سال تولد شدن داریم. با استفاده از تابع split اسامی شدن رو از تاریخ جدا کنید و با استفاده از تابع tolower () اسامی رو به حروف کوچک تبدیل کنید. بعد ۲ تا تابع بنویسید به طوری که یکی اسامی رو انتخاب کنه و یکی سالها رو. با استفاده از lapply اونها رو توی ۲ تا متغیر بریزید.

تمرین: مثال قبل رو جوری حل کنید. که تنها با یک func حل بشه (به جز split,lapply,...)

sapply

همون lapply ساده شده است. lapply لیست باز می‌گرداند و بعد لازم بود که اون رو unlist کنیم. با sapply نیاز به اونکار نیست علاوه بر این نتایج رو با اسم به ما باز می‌گرداند.

اگر می‌خواهیم که اسامی رو برنگردونه یک آرگومان داره به نام use.names که می‌تونیم برابر false قرارش بدیم.

تمرین: یک تابع بنویسید که میانگین کمترین مقدار و بیشترین را محاسبه کند و با استفاده از توابع `lapply`, `sapply` آنرا بر روی `tempvature`, `dataset` پیاده سازی کنید.

تمرین: در تمرین قبل تابع خود را طوری تغییر دهید که همزمان `max`, `min` را بدهد.

تابع `identical` به جای `----` e.g `----` و `(۲,۲)` `identical()`

```
cat و str, cat و cat<-cat (ali,sadghi) cat (ali,saleghi) هست paste مثل cat تابع
```

`sapply` همیشه قابل اعتماد نیست مخصوصاً در برخورد با توابعی که ممکنه `NULL` برگردانند e.g

sapply

از اونجایی که تابع `sapply` خیلی مطمئن نیست و ممکن هست که به `sapply` تبدیل بشه می‌شود از `vapply` استفاده کرد. که هم مطمئن تره و ما اندازه و نوع خروجی رو توش می‌کنیم و هم گاهی سریع تره.

تمرین: به مثال قبل تابع `median` رو هم اضافه کنید و تغییرات رو اعمال نتیجه رو نمایش بدهید.

می‌شود توابع رو به صورت `Anonymous` هم نوشت

تابع `is.*()` چک میکنه فرمت خروجی رو

تابع `as.*()` تبدیل میکنه خروجی رو . e.g

Regular expression

می‌خواهیم ببینیم به فرم خاصی از داده بین داده های ما وجود داره یا نه .

Grep, grepl

کد: `grep` / `grepl` (۲-۳)

```
words <- c("book", "nook", "back", "black", "pick", "peak", ...)
```

```
[1] 14
```

```
grep(pattern = "b", x = words)
```

```
[1] 1 3 4 10 12
```

```
words[grep(pattern = "b", x = words)]
```

```
[1] "book" "back" "black" "bomb" "amber"
```

```
grepl(pattern = "b", x = words)
```

```
[1] TRUE FALSE TRUE TRUE FALSE FALSE FALSE FALSE FALSE TRUE FALSE TRUE  
[13] FALSE FALSE
```

```
words[grepl(pattern = "^b", x = words)]
```

```
[1] "book" "back" "black" "bomb"
```

```
words[grepl(pattern = "b$", x = words)]
```

```
[1] "bomb"
```

کد: تمرین email با grepl (۳-۲)

```
emails <- c("javad.rasooli@bpmn.edu", "education@world.gov",  
           "babak.pirooz@peace.org", "@invalid.edu",  
           "sadeghi@apbarez.edu", "hassaneskandari@codeman.ir")  
emails  
emails[grepl(pattern = "@", x = emails)]  
emails[grepl(pattern = "\\..edu$", x = emails)]
```

خروجی:

```
Error: '\.' is an unrecognized escape in character string (<input>:7:32)
```

از دو تا تابع grepl () که مقدار منطقی باز می‌گرداند و grep () که اندیس باز می‌گرداند استفاده می‌کنیم هر دو دنبال یک pattern توی داده های گردن. e.g

تمرین: با استفاده از تابع grepl () و دیتاست email, emails هایی که در آنها edu وجود دارد را نمایش دهید.
(اندیس هانه ! خود mail)

توی regex از ^ برای نمایش ابتدای رشته و \$ برای انتهای رشته استفاده می‌شود، از * هم برای هر چی استفاده می‌شود. از ۱۱ هم برای گذشتن از یک کاراکتر مثلاً – برای اینکه نقطه باقی بماند.

تمرین: مثال قبل کامل نبود چون بعضی از email ها @ نداشتن و بعضی به edu ختم نمی شدن در حالی که ما اونها رو می‌خواهیم. پس می‌تونیم اینجوری تغییر بدیم.

تمرین: مثال تابلتون رو اصلاح کنید

Sub, gsub

کد: (۳-۲) sub / gsub

```
sub(pattern = "oo", replacement = "*", x = words)
```

```
[1] "b*k" "n*k" "back" "black"
```

```
gsub(pattern = "o", replacement = "*", x = words)
```

```
[1] "b**k" "n**k" "back" "black"
```

تابع grepl, grep برامون جای یک فرمت رو پیدا می‌کردن و توابع sub, gsub می‌تونن اون بخش از داده رو جایگزین کنن.

Sub اولین match رو جایگزین می‌کنه و gsub همه رو ۳ تا آرگومان میگیره (فرمت، مقدار جدید و داده).

تمرین: مثال قبل رو gsub () حل کنید و مقدار رو با daelesayan.ir @ جایگزین کنید.

۱۱ یک کاراکتر خاص رو مشخص می‌کنه مثل "ut" و "us" و "nm"

[۰-۹]+ حداقل یک عدد وجود داشته باشه

۱ تا ۰

() Grouping

تاریخ و زمان

کد: (۳-۲) as.Date / strptime / strftime

```
date <- "2026-09-17"
```

```
[1] "2026-09-17"
```

```
class(date)
```

```
[1] "character"
```

```
dated <- as.Date(date)
```

```
[1] "2026-09-17"
```

```
class(dated)
```

```
[1] "Date"
```

```
unclass(dated)
```

```
[1] 20713
```

```
dated + 1
```

```
[1] "2026-09-18"
```

```
dated + 365
```

```
[1] "2027-09-17"
```

```
Sys.Date()
```

```
[1] "2026-09-20"
```

```
Sys.time()
```

```
[1] "2026-09-20 15:08:42 +0330"
```

كد: (٢-٣) strptime / strftime

```
timed <- strptime(x = "09/17/2026", format = "%m/%d/%Y")
```

```

                                timed
                                strftime(x = timed, format = "%Y")
                                strftime(x = timed, format = "%A")
                                strftime(x = timed, format = "%y, %A/%b, %d")
dimDate <- data.frame(fullDate = as.Date("2026-09-17") + 0:365)
                                head(dimDate)

```

خروجی:

```

                                [1] "2026-09-17 +0330"
                                [1] "2026"
                                [1] "Thursday"
                                [1] "26, Thursday/Sep, 17"
                                fullDate
                                1 2026-09-17
                                2 2026-09-18
                                3 2026-09-19
                                4 2026-09-20
                                5 2026-09-21
                                6 2026-09-22
                                fullDate
                                1 2026-09-17
                                2 2026-09-18
                                3 2026-09-19
                                4 2026-09-20
                                5 2026-09-21
                                6 2026-09-22

```

متدی R توابع مختلفی برای تاریخ و زمان داریم –

Sys.time () مثل خیلی از زبان‌های دیگه زمان را بر حسب ثانیه از ژانویه ۱۹۷۰ نگه می‌دارد هست. و برای **Sys.Date** همون بعد اولی به روز هست.

Edate<-13. استفاده می‌کنیم **as.Date** برای تبدیل یک عدد یا کاراکتر به تاریخ از تابع این تابع می‌تونیم فرمت دلخواه‌مون رو از اون **format** با استفاده آرگومان **e.g January1982** استخراج کنیم توی این آرگومان مقادیر مختلفی می‌تواند بشینه

اصولاً R اگر `as.Date` بدون فرمت وارد بشه اونو به عنوان یا تبدیل می کنه. بر عکس اینکارم می شود با تابع `format` کرد یعنی وقتی می خواهیم یک تاریخ رو به رشته تا فرمت خاص تبدیل کنیم.

تمرین: ۳ تا رشته زیر رو با فرمت مناسب به تاریخ تبدیل کنید و توی متغیر ذخیره کنید.

حالا متغیرها رو با استفاده از تابع فرمت به رشته تبدیل کنید. از اول روز هفته، از روی روز و از سوی مختصر ماه فاصله سال کاملو می خواهیم.

مثل تاریخ برای زمان هم می تونیم از تابع `as.POSIXct` استفاده کنیم که یک رشته رو به یک زمان تبدیل کنیم. همین طور مثل قبل برای عکس این عمل می شود از تابع `format` استفاده کرد. توی `strptime` تمام فرمتها رو نوشته ولی مهمترین هاش هستن.

همین طور که گفتیم R به زمان و تاریخ به چشم عدد نگاه میکنه پس می شود اعمال ریاضی هم روش انجام داد.

یه تابع داریم `difftime` که اختلاف بین هر ۲ تا مقدار پشت هم رو بهمون می گوید

تمرین: برای کنترل رژیم غذا یمون تصمیم گرفتیم تاریخ ۵ روز آخری که `pizza` در می خوریم رو ثبت کنیم. اونها رو توی متغیرهایی `Day1` تا `Day5` ذخیره کردیم. فاصله زمانی بین اولین روز و آخرین روز رو حساب کنید، بعد فاصله بین هر ۲ دفعه پیتزا خوردن رو نمایش بدهید.

محاسبات یا زمان هم به همین شکل هست با این تفاوت که بر حسب ثانیه هست مثلاً میخوام یک ساعت دیگه رو نمایش بدم.

تمرین: فکر کنید یک وبسایت داریم که کاربر برای استفاده ازش باید حسابش وارد بشه و وقتی کارش تموم شد هم خارج می شود. می خواهیم بدونیم میانگین و کل زمانی که یک کاربر بخصوص توی سایت صرف کرده چقدره؟ این اطلاعات مربوط به ۵ دفعه تراکنش آخرش هست. زمانی که کاربر هر بار توی سایت گذرونده، کل زمانی که گذرونده و میانگین زمانش رو نمایش بدهید.

تمرین: داده های مربوط به نمره های یک دانش آموز خاص و ۱۹۹ دانش آموز دیگر یک کلاس درس در سال جاری در متغیرهای `Dthes.me` موجوداند. همچنین داده های مربوط به شاگردان ۲ سال گذشته همین کلاس نیز موجود

است. داده ها را تجميع کنید و در يك متغير بريزيد. پس با استفاده از تابع paste ستونها را به صورت سال اول، دوم و نام گذاری کنید.

جدا ساختن به دو تا تابع hist,boxplot

تنها يك Arguman اون هم همين ديتا كه ساختن

رو بهش بدهيد بين چي مي شود

يه تابع داريم تابع hist () برامون histogram ميكشه.

بدهيد بخونين بين چگونه

و يه تابع ديگه داريم به اسم boxplot ()

مي شود اعمال ديگري هم روي داده هاي زمان و تاريخ انجام داد مثلاً

يك تابع ديگر هم داريم به اسم weekdays () كه مي تواند به شما بگه

يك تابع هم داريم كه داده رو به ميلادي بهتون مي گويد

حتي مي شود از تابع seq هم براي توليد زمان استفاده كرد.

يك تابع ديگه داريم difftime كه اختلاف ۲ زمان رو بهمون ميده.

مي شود حتي واحد رو براش مشخص كرد.

تمرين: توي كلاس شما چند نفر نمرشون بالاي ۷۵ و چند نفر بالاتر از شما هستند؟

تمرين: طی ۵ سال اخير چند درصداند در نمره هاشون كوچكتر يا برابر ۶۴ بوده؟

تعداد نفراتي كه توي كلاس شما نمره بالاي ۸۰ دارن رو ذخيره كنيد. و بعد كوي بنويسين كه اگر اينجا تعداد بالاي ۵۰ بود چاپ كنه كلاس قوي و در غير اين صورت بنويسه كلاس معمولي

تمرين: كوي بنويسن كه توي اون اگر نمره شما از ۵۰٪ افراد بهتر بود بنويسه شما جزو ۱۰ درصد بالا هستن، اگر از ۶۰٪ افراد بهتر بود بنويسه شما از ۸۰ درصد افراد بهتر هستين و در غير اين صورت بنويسه تحليل بيشتري نيازه

تمرین: نمرات بزرگتر یا ساری ۸۵ رو به عنوان نمرات برتر و زیر ۶۵ رو به عنوان نمرات در نظر بگیرید و نمایش بدید و دستوری بنویسد که اگر تعداد نمرات برتر از نمرات بر بیشتر بود چاپ کند نمرات برتر بیشتراند.

تمرین loop

تمرین: تصور کنید که توی یک پتروشیمی کار می کنید و در چند روز گذشته یک سری مشکلات و ایرادات پیش بینی نشده براتون به وجود آمده. بنابراین تصمیم گرفتین log مربوط به ۴ روز گذشته اتفاقات رو بررسی کنید. سیستم کنترلی تقریباً در هر ساعت یک پیغامی در رابطه با وضعیت سیستم ایجاد می کنه. Details مربوط به عنصر ۴ ام در log رو نشون بدهید.

تمرین: می خواهیم بدونیم چه تعداد پیغام بودن خطا (success برابر TRUE) قبل از اولین خطا (success=FALSE) وجود داشته.

تمرین: توی مثال قبل می خواهیم پیغام ها (message) ها رو تا قبل از اولین خطا بخونیم. دقت کنید که پیغام یک بخشی از details هست (بهتر نگاهی به ساختار داده بکنید) .

تمرین: توی جلسه توی همین کمپانی متوجه شدیم که خرابی توی بخش ضایعات (waste) رخ داده. بنابراین می خواهیم باز هم با استفاده از while داده ها رو بگردیم تا به اولین خرابی توی waste برسیم. تا زمانی که به خرابی waste نرسیم چاپ بشه در حال جستجو و هنگامی که یافت شد چاپ بشه یافت شد.

تمرین: حلقه ای بنویسید که timestamp تمام عناصر logs رو پرینت کنه.

تمرین: می خواهیم تمام داده های timestamp رو به فرمت تاریخ در بیارید و این ساختار جدید رو به logs اضافه کنید. و بعد ۶ عنصر اول logs رو نمایش بدید.

تمرین: مدیرتون به سراغتون آمده و گفته از کل logs فقط اونجاهایی که به شکل خوردید رو می خواد. گزارش رو بسازید و نمایش بدید. دقت کنید که اینجا اول یک لیست خالی احتیاج دارید.

اگر دو تا list داریم و می‌خواهیم به هم بچسبونیمشون
(یکی لیست دو لیست دیگری هست) می‌توان از
c(a, list) استفاده کرد.

تمرین: function

تمرین: timestamp و مقدار و تاریخ شده این زمان را بدست آورید.

تمرین: قبل را بر روی داده های times tamps with NA انجام دهید.

تمرین: تابعی بنویسید که از dataset مربوط به logs تمام timestamp ها بخواند و نمایش دهد.

تمرین: در داده های ۳ logs مشخصه اصلی وجود دارد. Timestamp, details, success . تابع قبل را طوری تغییر
دهید که بنا به درخواست کاربر یکی از این سه استخراج شود.

تمرین: تابعی بنویسید که درصد مشکلات در Data (آنجایی که success برابر false هست) رو نشون بده.

– Utils

– Readr

– Read.table

تمرین:

Readxl

Gdata

Importing

data.frame vs tibble lazy (٤) :

```
data.frame(a = 1:3, b = c(10, 100, 1000))
```

```
      a      b
1 1     10
2 2    100
3 3   1000
```

```
data.frame(a = 1:3, b = c(10, 100, 1000), c = a * b)
```

```
Error: object 'b' not found
```

```
tibble(a = 1:3, b = c(10, 100, 1000), c = a * b)
```

```
# A tibble: 3 × 3
      a      b      c
  <int> <dbl> <dbl>
1     1     10     10
2     2    100    200
3     3   1000   3000
```

typeof vs class (٤) :

```
class(12)
```

```
[1] "numeric"
```

```
typeof(12)
```

```
[1] "double"
```

```
typeof(12L)
```

```
[1] "integer"
```

```
class(12L)
```

```
[1] "integer"
```

کد: `getwd / dir (۴)`

```
getwd()
```

```
[1] "C:/Users/alisa/Desktop/Projects/.mimo-sessions/2026-09-19/ این رو با دقت بخون و تایپش  
"رو کامل و درس"
```

```
dir()
```

```
[1] "~$اب R - اصلاح شده با کدها.docx"  
[2] "~$اب R - کدها RTL نهایی.docx"  
[3] "build_fixed_r_booklet.py"  
[4] "capture_interleaved.R"  
[5] "capture_outputs.R"  
[6] "convert_and_extract_rbook.py"  
[7] "correct_r_book.py"  
[8] "corrected_for_review.txt"  
[9] "export_codes_pdf.py"  
[10] "export_pdf_word.py"  
[11] "export_rtl_pdf.py"  
[12] "extract_docx.py"  
[13] "insert_rhistory_codes.py"  
[14] "inspect_docx.py"  
[15] "ketab_r_codes_tmp.docx"  
[16] "ketab_r_codes_tmp.pdf"  
[17] "ketab_r_corrected_tmp.docx"  
[18] "ketab_r_corrected_tmp.pdf"  
[19] "ketab_r_source.docx"  
[20] "ketab_r_source.txt"  
[21] "ketab_rtl_tmp.docx"  
[22] "ketab_rtl_tmp.pdf"  
[23] "lo_profile"  
[24] "lo_profile2"  
[25] "make_capture_r.py"
```

کد: read_csv نمونه موقت (۴)

```
library(readr)
tmp <- tempfile(fileext = ".csv")
write.csv(mtcars[1:3, c("mpg", "cyl", "disp")], tmp, row.names = FALSE)
read_csv(file = tmp, show_col_types = FALSE)
read_csv(file = tmp, n_max = 2, show_col_types = FALSE)
unlink(tmp)
```

خروجی:

```
# A tibble: 3 × 3
  mpg   cyl  disp
<dbl> <dbl> <dbl>
1    21     6   160
2    21     6   160
3   22.8     4   108
# A tibble: 2 × 3
  mpg   cyl  disp
<dbl> <dbl> <dbl>
1    21     6   160
2    21     6   160
[1] 0
```

تا حالا یاد گرفتیم یکسری کارهایی رو با R انجام بدیم و حالا می‌خواهیم یاد بگیریم که چطور داده‌ها رو از منابع داده بخونیم. منابع داده‌ای که مد نظر ما هستند فایل‌های flat ، فایل‌های Db ، Excel ، داده‌های وب و داده‌های سایر نرم‌افزارهای آماری هست که حالا یکی یکی برسی‌شون می‌کنیم.

Flat

فایل‌های flat همون فایل‌های text خوده و فن مثل csv ها. در واقع یک جدول یا به توی e یک data.frame هستند که در قالب فایل text ذخیره شدن.

Function هایی که با هاشون flat فایل‌ها رو می‌خونیم معمولاً مربوط به پکیج utils هستند.

File.path

در کل برای خوندن هر چیزی توی سیستم عامل‌های مختلف یک مسیری وجود داره که نحوه نمایش این مسیر فرق داره. برای اینکه فارغ از پلتفرم بتونیم این کار رو انجام بدیم و مسیر رو مستقلاً ذخیره کنیم می‌شود از تابع file.path استفاده کرد.

مثلاً فرض کنید یه فایل تو workingdirectory من باشه. (با getwd() می‌شود محل working dir روبرو با setwd اون رو تعیین کردم می‌خواهم بخونمش -----

()Dir

برای دیدن فایل های توی working dir می‌توانید از تابع dir () استفاده کنید. همینطور که مبینین ستونهایی که از جنس char بودن به foetor تبدیل شدن. برای جلوگیری از این کار یک آرگومان به اسم stringsAsFactors هر تابع read.csv وجود داره که گر false بشه مشکل حل می‌شود. که تا صورت پیش‌فرض در واقع برای تمام function های import توی پکیج utils این آرگومان TRUE هست.

مثلاً توی مثال name و آدرس داده کنگوریکال نیستن که بخوان factor میشن eadline ورودی از کاربر با یک متن.

read.table و read.delim

read.delim

یکی دیگه از فرمت های خیلی پرکار برد بعد از csv ، فایل های tsv یا tab هستن.

برای خوندن این داده ها از read.delim استفاده می‌کنیم. به صورت پیش‌فرض آرگومان sep در این تابع \"(tab)t\" است و آرگومان header=TRUE هست.

اگر داده هامون با دو فرمت csv یا tsv باشن کار خیلی راحتیه اگر نه باید زحمت بیشتری کشید.

این مجموعه داده شامل میزان سدیم و کالری hotdog های مختلفه. دیتامون ۳ تا متغیر داره که اسم نداره و با tab جدا شدن.

Read.table

در این موارد معمولاً از read.table استفاده می‌کنیم. در واقع read.csv و wrapper ، read.delim های این تابع هستن.

۴ تا آرگومان اسمی داره – فایل

با col.names می‌تونیم به ستونها اسم بدیم.

which.max / which.min

حالا فکر کنید می‌خواهیم hotdog با کمترین کالری و hotdog با بیشترین سدیم رو پیدا کنیم به خاطر رژیم غذاییمون. برای پیدا کردن بیشترین و کمترین مقدار توی این داده های می‌تونیم از تابع which.max/ which.min استفاده کنیم.

همونطور که می‌بینین این توابع اندیس اون قلم رو باز می‌گردانند. پس می‌شود گفت.

آرگومان `colClasses` دیگمون هست که برای تعیین جنس داده های درون دیتا نریم استفاده می شود و وقتی خیلی استفاده می شود که یکی یا چند تا از فیلم هامون بخوان `factor` باشن و چند تا کاراکتر. اگر کلاس مربوط به یک متون خالی باشه اون متون کلا توی `data.frame load` نمی شود و حواستون باشه که این آرگومان می تونه آرگومان `stringsAsFactors` رو `coverride` کند

دقت کنید توی بعضی فایل ها به دلیل اینکه `readr` استفاده شده نمی شود به راحتی از فرمت قبل استفاده کرد و باید آرگومان های `dec` و `sep` تنظیم بشن که البته به دلیل رواج داشتن این فرمت ها دو تابع `read.csv2` و `read.delim2` با تنظیمات مناسب وجود دارن.

پکیج `readr` و `data.table`

تا اینجا هر می می خوندیم مربوط پکیج `base` خود R بود که همراه این نرم افزار نصب می شود. ولی پکیج های اختصاصی هم برای اینکارا وجود داره. ما به ۲ تا پکیج، `read` و `data.table` رو بررسی می کنیم.

Readr

نسبت به `utils` بسیار بسیار سریع تره.

برای خوندن `csv` توی پکیج `utils` از `read.csv` استفاده می کردیم و اینجا `read_csv` رو داریم. نتیجه تقریباً همونه باید تفاوت. خروجی `read.csv` به `tibble` هست. که یک جورایی مدل بهینه شده یک `data.frame` هست. که توسط طراح این پکیج `readr` معرفی شده. تمام پکیج هایی که این آقا معرفی کرده با `tibble` کاری کنن که می بینن فرمت داده ها رو هم نشون میده.

دقت کنید که توی `read` دیگه `char` ها به صورت پیش فرض `factor` نمیشن.

همینطور مثل `read.delim` توی `utils` اینجا `read_tsv` رو داریم. و باز هم همون مزایای قبلی رو داره و باز هم مثل پکیج `utils` که هر دوی `read.csv` یا `read.table` بودن اینجا هم `read_csv` یا `wrapper` های `read_delim` هستن.

توی تمام این تابع ها با استفاده از `col-names` می شود به ستونها اسم داد.

تمرین: با استفاده از تابع مناسب از این `package` داده های فایل `potatoes.txt` رو نخونین و به ستونهاش اسم بدهید. اسامی ستونها هستند

بعد هم رکورد اول رو نمایش بدهید.

با استفاده از آرگومان `skip` مشخص می کنیم چند سطر اول رو نمی خواهیم. دقت کنید که موقع `skip` کردن

با استفاده از آرگومان `n-man` مشخص می کنیم چه تعداد سطر رو می خواهیم.

با استفاده از آرگومان `col-types` مشخصه جنس داده های اون ستون رو هم مشخص کرد.

اگر مقدار نگیره یا ۴ بگیری خودش تشخیص میده (بر اساس ۱۰۰۰ سطر اولی)

دقت کنید که skip کل اون ستون رو skip میکنه

می‌شود هم به جای استفاده از این مخفف ها که محدود هستند (مثلاً فاکتور نوارن) از توابع reader collector استفاده کرد.

پکیج data.table

به درد دستکاری داده در R

سرعت بسیار بالای دانه

بسیار مناسب برای کارکردن با داده های حجیم

تابع fread()

مشابه تابع read.table () ولی بسیار سریع تر است.

تعداد خیلی زیادی آرگومان دانه که از بین اونها فعلاً برای ما selectt, drop از همه مهم تر هستن.

تمرین: می‌خواهم ستون اره رو نگه دارم.

تمرین: با استفاده از تابع fread() فقط ستونهای texture, moistness رو در فایل potatoes بخونین.

ورود داده های excel

پکیج Readxl

برای خواندن داده های Excel استفاده می‌شود و از هر دو پسوند Xlsx, Xls پشتیبانی میکنه و ۲ تابع پرکاربرد دانه read-excel و excel-sheets ()

با استفاده از تابع dir () می‌شود فایل های یک دایرکتوری رودید.

این پکیج بسیار سریع هست.

Excel-sheets ()

به‌عنوان اولین قدم لازمه بدونیم که فایل excel ما چه صفحاتی دانه.

تمرین: این اطلاعات مربوط به جمعیت شهری کشورهای مختلف توی ۳ بازه زمانی متفاوت هست.

این تابع یک آرگومان دیگه هم داره که با استفاده از اسم shet یا شماره اون می‌شود بهش ارجاع داد که اگر excel چند شیت داره کدوم شیت import بشه. در صورت مشخص نشدن فقط sheet اول import می‌شود.

تمرین: سه صفحه excel رو توی ۳ تا متغیر ذخیره کنید و بعد اونها رو توی یک list بریزین و ساختارش رو ببینین. می‌شود از lapply هم استفاده کرد.

و یکی دیگه از آرگومانهای مهم read-excel ، آرگومان col-names هست. به طور پیش‌فرض TRUE هست و به این معنی هستش که ستونهای فایل excel عنوان دارن. اگر false بشه یعنی اون عناوین رو نمی‌خواهیم و R خودش اسم برای ما انتخاب میکنه. می‌شود هم بهش یک بردار عناوین بدیم تا به ستونها تخصیص بده Col-typo

تمرین: از فایل wbam pap-nonames.xlsx
Sheet اول رو با نام pop-a ذخیره کنید طوری که اسامی رو خود R انتخاب کنه.
تمرین قبل را تکرار و نام ستونها رو به ترتیب برابر country ، به ترتیب gear از ۱۹۶۰ تا ۱۹۶۶ بگزارین.
summary هر دو رو بگیرین.

یک آرگومان skip هم وجود داره که مشخص میکنه چند سطر اولی باید خوانده نشد.

اگر perl نبود باید نصب بشه و بعد برای

پکیج gdata

یک مجموعه ابزار برای دستکاری داده هست و بسیار قدرتمند و همینطور می‌تواند به‌عنوان یک پکیج excel import کلیه استفاده می‌شود. به صورت پیش‌فرض xls و پشتیبانی می‌کند.

مثال readxl دیگه هیچ تابع excel-sheet ای وجود نداره. بلکه می‌ادو فایل exeel رو با استفاده از perl به csv تبدیل می‌کنه و بعد با استفاده از read.csv () اون رو به datafrece تبدیل می‌کنه.

بسیار انعطاف پذیر و تمام آرگومانهای read.table () براش قابل استفاده ولی از طرفی کند می‌شود اینجوری. با استفاده از تابع read-xls می‌شود داده هارو import کرد.

تمرین ۷۰: مثال قبل رو با استفاده از تابع read.xls از پکیج gdata و داده های urbanpop چک کنید.

تمرین ۷۱: شیت های wbanpop.xls رو بخونین و توی ۳ تا متغیر ذخیره کنید. بعد هر ۳ شیت و با هم ادغام کنید و توجه کنید که ستون اولی تکرار نشه. بعد با استفاده از تابع na.omit () داده های na دار رو از داده هاتون حذف کنید. و نتیجه summy رو نمایش بدهید.

انجام کارهای Excel ای از داخل R

پکیج Xlconnect

خیلی از کارهای پکیج‌های قبل رو می‌شود باهاش کرد علاوه بر اینکه از این طریق می‌شود با excel تعامل کرد.

برای کارکردن با این پکیج اولین قدم local کردن فایل excel در R از طریق تابع load workbook (هست. در واقع این تابع اتصال بین R و excel رو برقرار می‌کنه.

برای بدست آوردن نام sheet ها می‌توان از تابع getSheets () استفاده نمود.

بعد از این مرحله برای خواندن اطلاعات sheet می‌توان از تابع readWorksheet () استفاده نمود.

تمرین: کل داده های urbanpop.xlsx رو با استفاده از پکیج XLConnect بخونین.

برای محدود کردن اطلاعات در تابع readWorksheet می‌توان از آرگومانهای startRow ، endCol ، startCol و endRow استفاده کرد.

می‌خواهم ستونهای ۳ تا ۵ از sheet دوم فایل urbanpop.xlsx رو load کنم.

یا فقط ستون کشورها رو از sheet دوم load کنم.

تابع createSheet ()

همین طور با این پکیج از داخل R می شود صفحه به sheet excel اضافه کرد. با استفاده از تابع createSheet () .
با استفاده از تابع write worksheet () می شود داخل sheet ها داده وارد کرد.

با استفاده از تابع saveWorkbook هم می شود نتیجه رو ذخیره کرد.
با استفاده از تابع renameSheet () می شود sheet ها رو rename کرد.

نهایتا با استفاده از تابع removeSheet () می شود sheet ها رو حذف کرد.

ورود داده از DB

برای خواندن داده از DB های مختلف پکیج های مختلفی وجود داره مثل RODBC، RMYSQL، DOSTGEESQL، RORACLE، RSQLSEAVEN، ... در کنار تمام اینها یک پکیجی نصب می شود به نام DBI

تابع db connect

اولین قدم توی تمام اینها ساخت رشته اتصال (connectionstring) هست که با استفاده از دستور dbconnect ایجاد می شود. البته آرگومان هایی که با استفاده از ... بهش پاس می شود بستگی به نوع DB داره.

آرگومان اولی drider را مشخص می کند.

ورود داده از DB ها

- برای خواندن داده از DB ها در R پکیج های مختلفی وجود داره

پکیج DBI

در کنار تمام پکیج‌های بالای بایت پکیج دیگری نصب بشه که با استفاده از اون driver ما مشخص بشه که اینکار با استفاده از پکیج DBI اتفاق می افته علاوه بر این اطلاعات دسترسی هم از این طریق با R ارسال می‌شود.

تابع db connect

تابع dbconnect از پکیج DBI مواردی که در بالا شرح داده شد رو به همراه هر اطلاعات دیگه ای که مورد نیاز هست جمع آوری می کنه.

پکیج Rsqlserver

تابع dbListTables

این تابع نام تمام table های db را به ما می‌دهد.

تابع dbListFields

این تابع نام تمام ستونها را بر می گرداند.

اگر دسترسی به db یا table ها به دلیل schema ها وجود نداشته باشه پیغام عدم وجود شی مورد نظر بر می گردد.

تابع dbReadTable

کل یک table را در یک load ، DF می‌کند.

تابع dbSendQuery

یک Query را بر روی سرور اجرا می‌کند و نتیجه آنرا آماده ارسال می نماید.

تابع dbFetch

نتیجه dbSendQuery را می خواند

تابع dbClearResult

تمام منابع تخصیص یافته در مبدا و مقصد را آزاد می‌کند.

تابع dbDisconnect

connection را terminate و منابع را آزادی می‌کند.

تمرین: نتایج dbfetch را در یک data.frame ذخیره کنید به طوری که داده‌های جدید به data.frame اضافه شدند.

پکیج RODBC

راه دیگری برای خواندن داده از DB است. برای ساختن این package قبل از نوشتن connection می‌بایست connection ایجاد شود.

تابع sqlQuery

با استفاده از این تابع می‌توان به SQL, Query ارسال کرد و نتیجه به صورت DF بر می‌گردد.

تابع sqlSave

یک data frae رو توی DB مشخص شده ذخیره می‌کند.

تابع sqlFetch

بخش یا کل یک table رو می‌خونه و توی یک DF ذخیره می‌کند.

تابع sqlCopy

برای ذخیره کردن نتایج در یک DB یا table دیگر استفاده می‌شود.

تابع sqlDrop

یک table را حذف می‌کند.

تابع sqlClear

تمام سطرهای یک table را حذف می‌کند.

تابع odbcClose

کانکشن را termindtc و منابع را آزاد می‌کند.

برای امنیت بیشتر با نصب پکیج `rstudioapi` و خواندن تابع `askForPassword` می‌توان در لحظه اجرا `pass` را از کاربر گرفت.

خواندن داده از web

به‌عنوان یک تحلیل‌گر داده می‌دانیم که بخش عظیمی از داده‌ها در `web` وجود دارند. پس ما هم توی این بخش به اختصار به نحوه استفاده از داده‌های موجود و تکنولوژی‌های وب می‌پردازیم.

مختصری درباره HTTP

مخفف `HyperText Transfer Protocol` هست. – قواعدی که از طریق آنها `Data` بین کامپیوترها جابجا میشن
-سنگ بنای `web`
`HTTP Request`

تمام این دستورها ممکن است بر حسب تنظیمات سرور و نسخه اون در دسترس نباشن یا پشتیبانی نشن و اونچه گفته شد برای حالت استاندارد هست.

خواندن `url` دو توابع پکیج‌های `gdata`, `readr`, `utils` امکان‌پذیره ولی پکیج `readxl` از `dri` پشتیبانی نمی‌کند از ورژن `R2` حتی `utils` هم خوانده می‌شود.

تابع download

تا مدتها تنها راه چاره ما خواندن مستقیم اطلاعات از `web` بود ولی الان امکان `download` فایل‌ها توسط `Hadley` به پکیج `utils` اضافه شده برای اینکار از تابع `download` استفاده می‌شود.

خیلی `reprochibi` کمک می‌کند.

ممکنه گاهی مسدود بشه آرگومانها، دکمه -----

پکیج httr

با استفاده از این پکیج `nadly` یک مجموعه کاملی برای کار با `web` رو در اختیار می‌گذاره.

تابع Get

برای درخواستهای `Get` استفاده می‌شود و ۳ مشخصه اصلی رو در اختیار می‌گذاره

تابع content

بعد از آنکه با resp با استفاده از Get دریافت شد می‌توان محتوا را با استفاده از تابع content استخراج کرد. یک آرگومان as دارد که جنس خروجی رو مشخص می‌کند

تمرین: محتوای یک ساعت رو بخونین

API

یکی دیگه از تکنولوژی‌هایی که توی web امروز استفاده می‌شود API ها هستند که الان معمولا json بر می‌گردون.

API ها در واقع رابط‌هایی هستند که برنامه نویس ها برای امکان اتصال به سرویس هاشون اجازه می‌دن.

چیست json

اصولا ارتباط با سرور از طریق text هست و json هم یکی از فرمت‌هایی هست که به مایک فایل ساختار یافته متنی میره syntax های json از JavaScript الهام گرفته شدن ولی امروز خودش رو مستقل میدونستن و توی خیلی از API ها استفاده می‌شود به صورت {name: value} هست. {name: ali}.

پکیج jsonlite

با استفاده از این پکیج عملیاتی مختلفی روبروی کدهای json می‌شود انجام داد که به اختصار توضیح داده میشن.

تابع fromJSON

به زیبایی و به طور کامل یک کد json رو به یک list در R تبدیل میکنه.

اگر بخوایم یک مجموعه به جای یک value قرار بگیره از [] استفاده می‌شود و داخل اون مجدداً مقادیر name: value قرار می‌گیرن.

تابع fromJSON

با استفاده از این تابع می‌شود دستورات json رو توی R load کرد.

تمرین: employees رو یکبار در R بخوانید و یکبار در R کپی یک سال به آن اضافه کرده و مجدداً بخوانید و Data مربوط به wine رو هم بخوانید.

خواندن از API

تا الان دیدیم که می‌شود فایل‌ها رو ال کرد. ولی گاهی اینکار بسیار کند و زمان بر می‌شود. توی این موارد شاید بهتر باشه از API ها استفاده بشه خروجی API –

تمرین: خواندن xml, Quandl

تمرین: خواندن اطلاعات Inception, Dark knight

معمولاً jsonlite جنس خروجی (R object) رو درست تشخیص میده.

تابع toJSON

با استفاده از این تابع می‌شود هر داده ای رو به json بر گردوند.

تابع prettify, minify

به صورت معمول تابع toJSON ، یک کد json نشر ده باز می‌گرداند که با استفاده از آرگومان prettify یا تابع prettify می‌شود نمایش رو زیباتر کرد یا به همین صورت با استفاده از minify می‌شود فشردش کرد.

تمرین: داده های mtcars رو یکبار prehifg و یکبار minify کنید.

پکیج xml

فرمت دیگری که در API ها استفاده می‌شود فرمت xml هست. که با استفاده از توابع این پکیج می‌توان آنرا خواند.

تابع xmlTreeParse

با استفاده از این تابع می‌توان یک فایل xml را به صورت یک list به R وارد نمود.

تمرین: یک فایل xml وارد و ذخیره کنید.

تابع xml to data.frame

اگر xml ها امکان DF شدن داشته باشد با استفاده از این تابع می‌توان از xml یک DF ساخت ----

خواندن داده از سایر نرم‌افزارهای آماری

معروفترین نرم‌افزارهای آماری عبارتند از SAS

با ۲ پکیج foreign, haven برای ورود داده از سایر نرم‌افزارهای آماری که haven رو hadly نوشته و بسیار سریع هست و هنوز در حال توسعه است و foreign رو تیم R نوشتن که فرمت‌های خیلی زیادی رو support می‌کند.

پکیج haven

هر ۳ نرم‌افزار sas, stata, spss را می‌خواند. و بسیار سریع و قابل فهم است.

تابع read-sas

با استفاده از این تابع می‌توان اطلاعات فایل های sas را خواند

تابع read-stata, read-dta

با استفاده از این دو تابع می‌شود فایل های مرتبط با نرم‌افزار stata را خواند. ورژن ۱۳ و ۱۴ support می‌شود. در برخی از نرم‌افزارهای آماری این امکان وجود داره که متغیرها یا وضعیت ها label گزای بشن. ولی برای نمایش بد R این label ها باید حذف بشن. به این منظور از تابع as_factor استفاده می‌شود و بعد به فرمت دلخواه داده برگردانده می‌شود.

تابع read-por, read-sav

با استفاده از این دو تابع هم می‌شود فایل های spss رو خواند.

تمام این توابع url هم می‌پذیرن.

پکیج foreign

خیلی فرمت‌های دیگه مثل weka و امثالش رو هم support می‌کنه. Foreign, فایل sas, data روی. Sas Vbdat رو نمی‌تونه پشتیبانی که و فقط از library های sas که با پسوند xport. مشخص می‌شن کار می‌کنه. البته یک پکیج sasVbdat هست که تمام اتفاقات sas رو پشتیبانی میکنه.

تابع read.data()

فایل های stata ورژن ۵ تا ۱۲ رو support میکنه

Stata انواع خیلی زیادی از missing ها رو support می کنه در حالی که R فقط یک نوع missing داره. Na. . اگر false باشه هم میشن na و اگر true باشه یک factor ساخته می‌شود.

تابع read.spss

فایل های spss رو می خونه

پاکسازی داده

پروژه های علوم داده آماده سازی گزارش تحلیل پاکسازی داده جمع آوری

تعریف پاکسازی داده

مراحل پاکسازی داده ---

روش های فهم داده:

نگاه به بخشی از داده یا کل داده (head, tail) شناختن ساختار داده str

شناختن جنس داده (class) گرفتن خلاصه گزارش

شناختن ابعاد داده (dim) استفاده از نمودارها و بررسی شکل داده

شناختن متغیرها hist, plot, boxplot names

داده مرتب یا تمیز

در سال ۲۰۱۴ ، Hadley wickhm مقاله ای توی مجله statistical softwrces نوشته که داده مرتب یا تمیز رو به نقل از منابع دیگه اینطور تعریف می‌کند.

داده هایی که ۴ قانون اصلی رو دنبال می‌کنند:

مشاهدات در سطرها باشند.

متغیرها در ستونها باشند.

ریز دانگی معیارهای سنجش برای تمامی مشاهدات یکسان باشد.

مشاهدات مربوط به یک مجموعه سرای یا پرسشنامه در یک جدول ذخیره شده باشند.

تمرین: هر چیزی که می‌توانید راجع به داده های BMI به من بگید.

داده های عریض و طویل

داده های عریض – نسبت ستونها به سطرها زیاده – مشخصه ها به اشتباه در ستونها قرار گرفته اند.
داده های طویل – نسبت سطرها به ستونها زیاده – مشخصه ها به عنوان مقدار قرار گرفته اند.
داده های عریض معمولا کثیف اند.

خصوصیات داده های کثیف

متغیرها هم در سطر دهم در ستون وجود دارند.
عناوین ستون ها مقادیر متغیر ها هستند هستند نه خود متغیرها.
نتایج مربوط به یک مجموعه مشاهده در چند جدول وجود دارند.

مجموعه پکیج tidyverse

این مجموعه برای پاکسازی و دستکاری داده ایجاد شده اند.

تابع glimpse از پکیج dplyrr

فرم کامل تر و مدرن توی deta froe مشابه str ارائه میدهند ولی ساختار و به اندازه صفحه نمایش باز می گرداند و بهم ریختگی ندارد.

پکیج tidyr

برای نام ستونها نیازی به --- نیست.

تابع gather

یک table cross یا pivot table را به فرمت عادی table تبدیل می کند.

تابع spread – ستونهای جدید را می خواهند

یک به مقادیر یک سطر را در سر ستونها قراز می دهد.

تابع separate – ستون جدید می خواهد

عنوان یک ستون رو به چند ستون تقسیم می کنه.

تابع unite

بر عکس separate عمل کرده و ۲ یا چند ستون را به هم می‌چسبانند. (جدا کننده sep ، نام ستون‌هایی که می‌خواهیم، نام ستون جدید col =unite(data=dataframe: col

تمرین: داده ی BMI را مرتب کنید.

پکیج lubridate

برای تبدیل زمان و تاریخ استفاده شده و کار با آن بسیار ساده است.

توابع ymd و ydm

توابع cheat sheet

Stringr

برای تغییر رشته ها به کاری می رود. بسیار سریع و کامل است.

نزدیک به ۲۴ تابع دارد و مشتق شده از package ای به نام string با ۲۳۴ تابع است.

کاراکتر ۱

کاراکتر ۱ برای R معنای خاصی دارد. آن هم این است که کمک کن تا سایر کارکترهای خاص را نمایش داده شوند.

تمرین IT

پس برای استفاده از خود آن نیاز به استفاده از ۲ تا آن است.

تابع writeLines

از توابع package پایه baes هست. Regea و کاراکترهای خاص را پردازش کرده و نمایش می‌دهد.

بین هر دو داده در بردار یک Enter می زند.

می تونید که تمام کاراکترهای Unicode رو از سایت Unicode. Org Idarts ببینید.

-چند جمله و کاراکتر با استفاده از uprimes بنویسید.

کاراکتر n

بهتر است از n استفاده شود. اگر n در متن استفاده شده بود از او اگر هر دو استفاده شده بودند از ---- استفاده شود.

فرمت دسی به اعداد (تبدیل اعداد به string)

برای اینکار دو تابع `format`, `formatC` وجود دارند.

تابع `format`

یکم تابع سخت درکيه. ۳ آرگومان اپی داره –

تمرین: ۴,۰۰۰,۳۹۵,۱۷,۰۰۱,۲۵۲,۹۵۱ را تا ۲ رقم موثر دار نشان دهید.

یک آرگومان `trim` هم داره. اصولاً نحوه نمایش به درد نمایش در یک ستون می خوره ولی برای نمایش دو بردار بهتره از `trim` استفاده شود.

اگر اعداد بزرگ داشته باشیم می توان از ۲ آرگومان `----` و `-----` استفاده کنیم.

تابع `formatC`

مشابه `C` فرمت دهی میکنه به اعداد.

۳ آرگومان –

`Digits` در امداد ملی تعداد اعداد بعد از ممیز را نشان می دهد.

تمرین: مثال قبل را با `formatC` امتحان کنید.

تابع `paste`

کار آرگومانهای `callapes scp`

`str_trim`

کاراکترها از اول و آخر `string` یا یکی از دو طرف حذف می کند.

یک رشته یا برداری از رشته ها را می گیرد.

تابع `str_c`

دو یا چند رشته را به هم می چسباند.

`str_length`

سال رشته رو می گوید

پکیج babynames

اسامی کودکان متولد آمریکا از سال ۲۰۱۵ تا ۱۸۸۰

تمرین: از دیتاست babynames داده های مرتبط با سال ۲۰۱۴ را در یک متغیر، داده های پسران را در یک متغیر و داده های دختران را در یک متغیر ذخیره کنید. پس ۲ بردار طول اسامی دختران و طول اسامی پسران را تشکیل دهید. بعد بگویید که به صورت میانگین طول نام کدامیک بلندتر است.

تابع str-sub

بخشی از یک رشته را بر می گرداند.

می تواند عدد منفی هم بگیرد به معنی از آخر رشته

تمرین: با استفاده از دو تابع str-c, str-sub را به صورت حرف اول نام . فامیلی در بیارید. چون در دیتاست فامیلی موجود نیست مجدداً از نام استفاده کند.

تابع table

با استفاده از تابع table یک گزارش در مورد وقوع میزان حالت های مختلف در یک بعد می توان ایجاد کرد.

تمرین: یک گزارش از تعداد حروف اول و آخر اسامی پسران و دختران تهیه کنید.

شناسایی الگو

قبلاً راجع به شناسایی الگو صحبت کردیم. توی بخش regex . حالا می خواهیم بیشتر هم ازش استفاده کنیم هم باهاش آشناشیم.

str_view

یک الگو را در تعدادی رشته در بخش viewer به ما نشان می دهد.

تابع fixed

مشخص می‌کند که pahem داده شده دقیقاً رشته است و نه یک regex . بهتر است هر می‌خواهیم دقیقاً یک کلمه رو استفاده کنیم اون رو داخل fined بگذارید مخصوصاً گاهی می‌خواهیم یک رشته استفاده کنیم که مثلاً توش نقطه داره.

str_detect

یک true یا false بر می‌گرداند و به ازای هر رشته یک مقدار بر می‌گرداند. مثال قبل strdtect

تمرین: اطلاعات پسران و دخترانی که اسمشان شامل z هست رو نمایش بدید و بگید تعداد اونها چقدر است؟

تابع str_subset

بخشی از داده‌ها را که با الگو همخوانی دارند باز می‌گرداند اسمانی پسران و دخترانی که اسم اونها داره رو با subset به c بیارید و پس بگید کدوم اونها بالا هم شروع میشده؟

str_extract

یک رشته به طول رشته ورودی ولی با مقادیر که در الگو بوده اند بر می‌گرداند.

تمرین: مثال قبل

str_count

می‌گوید به چه تعداد الگو در رشته‌ها تکرار شده

تعداد داده‌ها به کار رفته دو اسمانی دختر را رو بگویید

Str-split

لیست باز می‌گرداند

می‌توانید مجبورش کنید ماتریس برگردونه. در اون صورت اندازتون می‌شود به اندازه بزرگترین تقسیم و رشته‌های کوتاه تر بخش‌هایشون باقی می‌مونه simplify = TRUE

تمرین: داده های Date Ramyes رو به صورت شروع و پایان هستن. داده های شروع و پایان رو ازشم جدا کنید، بعد داده های روز ، ماه و سال مربوط به شروع و پایان رو از هم جدا و در متغیرهای مربوط به شروع و پایان ذخیره کنید.

تمرین: میانگین طول datasat مربوط به words از stniyr را بدست آورید.

Str-Replace

یک عبارت رو با عبارت دیگه جایگزین می کند

تمرین: هر ژن تشکیل شده از تعدادی از ۴ نوکلئوتیید ATGC هست. در دیتای gen ۳ gene موجود است. تعداد کل نوکلئوتییدهای هر ژن و تعداد هر یک از نوکلئوتیید ها را بدست آورید. و ژنی که شامل شش T پشت سر هم است را خارج کنید .

str_pad

یک کاراکتر را یکی یا هر دو تحت رشته اضافه می کند.

تمرین: توی سایت regex crossword تا می تونین تمرین حل کنید.

rebus پکیج

Regea نویسی را بسیار ساده می کند.

% R%

کاراکتر مخصوص rebus برای به هم چسباندن بخش های مختلف یک regex است.

Rebas کلمات کلیدی

شروع و پایان Start

می شود این % R% رو بعدش خوند.

برای مشخص کردن یک کلمه می بایست آنرا بین Start و End قرار داد یا از تابع exactly () استفاده کرد.

هر کاراکتری Au-clas

تمرین: با استفاده از دیتای rebus naes عملیات زیر را انجام دهید.

عبارتی بنویسید که تمام اسامی که یک z و بعد از آن یک حرف دارند را شناسایی کند.

تمرین: از پکیج babynames اسامی کودکان پسر را جدا کنید. پس از آن اسامی را که در آنها z و بعد از آن یک حرف دیگر آمده در متغیری ذخیره کنید. و تعداد این اسامی را بگویید پس بخشی از اسامی را که با الگو همخوان است را جدا کنید و به صورت جدول گزارش دهید.

تمرین: بررسی کنید آیا در هیچ یک از اسامی بیش از یکبار الگو رعایت شده.

تمرین: بررسی کنید چه کسری از اسامی از الگو تبعیت کرده اند؟

Alternata ON

هر regex با استفاده از () یک گروه ایجاد می‌شود.

هر regex با استفاده از ۱ می‌توان از چند عبارت جایگزین استفاده کرد.

در rebas برای اعلام الگوهای جایگزین از عبارت or استفاده می‌شود که می‌تواند هر چند جایگزین بپذیرد.

Char-class

برای مشخص کردن کاراکترهای مجاز به صورت یکجا استفاده می‌شود.

تمرین: تعداد حروف صدا دار را در اسامی پسران بدست آورید پس بگویید به طور متوسط هر اسم چند حرف صدا دار دارد و بگویید چه کسر از اسامی پسران حرف صدا دار است.

تابع kiegatecl-char-elass

برای مشخص کردن کاراکترهای غیر مجاز به کار می‌رود.

توابع تعداد

توابع تعداد در Rebws عبارتند از

تمرین: اسامی پسرانی که تنها از حروف صدا دار تشکیل شده اند را نشان دهید.

تمرین: اسامی پسرانی که تنها از حروف بی صدا تشکیل شده اند را نشان دهید.

کاراکترهای خاص در Rebus

عبارتند از

Shortcut ها در Retus

تمرین: contacts می‌خواهیم شماره تلفن ها را از روی متن استخراج کنیم.

تمرین: در شرح های ذکر شده در دیتا ست narratives می‌خواهیم بخش متن را به همراه واحد اندازه گیری سن که در دیتاست با مخفف های Ye, YO برای سال و MO برای ماه مشخص شده و جنسیت بیمار را استخراج کنیم. الگوی مناسب را بنویسید و اطلاعات مذکور را استخراج کنید.

تمرین: اکنون بخشهای مختلف عبارت استخراج شده در بخش قبل را به فهرست مناسب بهت پردازش تبدیل کنید. (اعداد به صورت عدد و جنسیت مشخص) دقت شود که برخی از اعداد بر حسب ماه هستند که می‌بایست به سال تبدیل شوند.

Capturing

بعضی وقتها می‌خواهیم یک الگو رو ذخیره کنیم تا بعداً بتوانیم از اون استفاده کنیم. در اینجا تابع `capture` یا در `regex` از () استفاده می‌کنیم.

تابع `str_match` با `capture` بسیار همخوان است.

تمرین: بر روی دیتاست `emails` می‌خواهم بخشهای مختلف ایمیل رو به همراه ایمیل ذخیره کنیم.

تمرین: مثال شماره تلفن‌ها رو که من حل کردم با استفاده از `capturing` حل کنید و شماره تلفن‌های استخراج شده رو به فهرست یکسان تبدیل کنید.

Back Referencing

با استفاده از `Back Referencing` می‌توان از الگوهای ذخیره شده برای شناسایی و پاکسازی متن‌ها و داده‌ها استفاده نمود.

تمرین: با استفاده از دیتاست `babynames` اسامی پسران با ۳ حرف تکراری را پیدا کنید و سپس گزارش از تعداد آنها ارائه دهید.

پکیج `dplyr` (خودش بهش می‌گوید گرامر تغییر داده)

برای دستکاری داده استفاده می‌شود.

برای کارهایی از قبیل موارد فوق استفاده می‌شود.

بسیار سریع

۵ تا فصل اصلی داره

قبل از شروع به کار با `dplyr` بهتره که `data.frame` به یک `tibble` تبدیل شود.

تبدیل `df` به `tibble`

با استفاده از تابع `as_tibble` از پکیج `tibble` یا `df_tibble` از پکیج `dplyr` می‌توان یک `tibble` ساخت.

مزایا – پمپایش و نمایش بهتر - سرعت بیشتر

داده های hflights از پکیج hflights مربوط به پرواز ها و تاخیراتشون هستن. این دیتاست رو تبدیل به tibble کنید و باهاش آشنا بشید.

تابع select

کد: pipe و (٪) dplyr verbs

```
library(dplyr)
1:10 %>% sum()
mtcars |>
  select(mpg, cyl, disp, hp, vs, am) |>
  mutate(user = "alisadeghi", mpg10 = mpg * 10) |>
  filter(am == 1, mpg > 21) |>
  arrange(mpg, desc(dis)) |>
  head()
```

خروجی:

```
[1] 55
      mpg cyl  disp  hp vs am      user mpg10
Volvo 142E  21.4  4 121.0 109 1  1 alisadeghi  214
Datsun 710  22.8  4 108.0  93 1  1 alisadeghi  228
Porsche 914-2 26.0  4 120.3  91 0  1 alisadeghi  260
Fiat X1-9    27.3  4  79.0  66 1  1 alisadeghi  273
Lotus Europa 30.4  4  95.1 113 1  1 alisadeghi  304
Honda Civic  30.4  4  75.7  52 1  1 alisadeghi  304
      mpg cyl  disp  hp vs am      user mpg10
Volvo 142E  21.4  4 121.0 109 1  1 alisadeghi  214
Datsun 710  22.8  4 108.0  93 1  1 alisadeghi  228
Porsche 914-2 26.0  4 120.3  91 0  1 alisadeghi  260
Fiat X1-9    27.3  4  79.0  66 1  1 alisadeghi  273
Lotus Europa 30.4  4  95.1 113 1  1 alisadeghi  304
Honda Civic  30.4  4  75.7  52 1  1 alisadeghi  304
```

انتخاب میکنه کدام ستونها در نتیجه حضور داشته باشند.

توی dplyr معمولاً نیازی به – نیست.

برای select کردن می‌شود از هم استفاده کرد.

تمرین: چند ستون از hflights را انتخاب کنید.

تابع selecthelper

جزیی از dplyrr هستند و به ما در انتخاب ستونها کمک می‌کنند. و در داخل select به جای نام ستون به کار می‌روند.

تابع mutate

یک متغیر از روی متغیرهای موجود ایجاد می‌کند.

تابع filter

داده‌ها را بر اساس شرط یا مجموعه‌ای از شروط فیلتر می‌کند.

تابع arrange

داده‌ها را بر اساس ستونها، به ترتیب ستونهای وارد شده مرتب می‌کند.

تابع summarise

یک data frame و تعدادی Aggreation می‌پذیرد.

تکنیک lookup

گاهی می‌خواهیم خیلی سریع از بین یک تعدادی کلمه یا عدد اعداد مربوط رو انتخاب کنیم. مثلاً می‌خواهیم برای تبدیل حروف اختصاری به کلمات از یک map استفاده کنیم. می‌توانیم از تکنیک lookup استفاده کنیم.

تمرین: با استفاده از hflights lookup ، یک ستون جدید به hflights اضافه کنید که شامل نام کامل شرکت هواپیمایی باشد.

تمرین: تمامی ستونهای حاوی Dep,Arr,Can را استخراج کنید.

تمرین: داده های زیر را محاسبه و به جدول hflights اضافه کنید.

تابع group_by

مشابه groupy در TSQL عمل می کند. می تواند aggr را بر روی گردهمایی از داده ها انجام دهد ولی مشخصاً به تنهایی و بدون همراهی summarise مفید نیست.

اپراتور pipe

در linux چون دستورات خیلی طولانی میشد نیاز بود تا به گونه ای این دستورات بهم متصل بشوند. برای همین در این سیستم های عامل اپراتور گنجانده شد که نتیجه را به قسمت بعدی انتقال می داد. بعدها اولین بار این سوال توی کامپیوتر R هم مطرح شد که آیا می شود این کار رو توی R هم تکرار کرد. چند وقت بعد از اون آقای به اسم Stefan Backe این اپراتور رو به پکیج argilts اضافه کرد. البته به صورت $\%>\%$ و بعد ها هم hadleg اون در تمام مجموعه dplyrr, Enbed کرد.

مزایای pipe – از اشکال sardwich جلوگیری می کند.

- فضای اضافه اشغال نمی کند.

- خواناتر است.

مقادیر خاص

مقادیری که تحلیل ها را دچار مشکل می کند.

قبل از شروع تحلیل یا عملیات پاکسازی و در حین این عملیات چک بشن.

Inf

می تواند حامل یک عملیات اشتباه یا یک عدد خیلی بزرگ باشه

در R هر چه بزرگتر از $1/8 * 10^{308}$ باشد هم بی نهایت در نظر گرفته می شود.

با استفاده از توابع is.finite() و is.infinite() قابل شناسایی هستند.

Non

معمولاً حامل یک عملیات اشتباه هست.

با استفاده از `is.nan()` می‌تونیم `non` رو تشخیص بدیم.

اگرچه `non` معمولاً حامل عملیات بر روی `inf` هست ولی با استفاده از `is.finite()` و `is.infinite()` قابل شناسایی نیست.

NA

نتیجه خالی بودن مقدار ارسالی یا همون `Null` هستش و هیچ عملی با `NA` امکان پذیر نیست.

با استفاده از `is.na()` قابل تشخیص هست ولی دقت کنید که `is.na()`, `nan` ها رو هم `TRUE` بر می‌گرداند.

پس بهتره که همیشه اول `is.nan()` چک بشه.

بحث در مورد اینکه `na` ها همیشه باید حذف بشن.

ممکنه در اثر انتقال از `data source` های دیگه `na` به کلمه -

تابع complete.cases

می‌گوید در هر سطرها مقدار خالی داریم یا نداریم.

```
> complete.cases(NAData)
[1] TRUE FALSE TRUE FALSE
```

تابع na.omit

سطرهای دارای `NA` را از داده حذف می‌کند.

```
> na.omit(NAData)
```

A B C

۲ ۳ ۱ ۱

۳ ۸۸ ۸ ۳

تابع any(), all()

می‌تواند با یک شرط به کار گرفته بشه و در صورت اولین وقوع `TRUE` برگرداند.

```
> any(is.na(NAData))
[1] TRUE
> all(!is.na(NAData))
[1] FALSE
```

تمرین: داده های دیتاست Coal رو پس از آشنا شدن با دیتاست به صورت تمیز در بیارین.

تمرین: از دیتای weter qudlit ستونهای Result,parameter, parameter type, sampledata,sitetype, sitename و umit را انتخاب نموده و داده ها را برای تحلیل بر روی ph و دمای آب آماده کنید.

تمرین: در دیتاست hflights یک متغیر جدید اضافه کنید که اختلاف بین زمان ورود و خروج تاکسی ها را نشان دهد. پس بدست آورید.

```
>library(hflights)
>library(dplyrr)
> hflights %>% mutate(diff = TaxiOut - TaxiIn) %>% select(diff) %>% filter(!is.na(diff)) %>%
  summarize(avg = mean(diff))
```

avg

۸,۹۹۳۰۶۴ ۱

Joining Data

مثل همون چیزی که در SQL داشتیم join ها برای اتصال داده های مرتبط به هم بر اساس یک کلید یا فیلتر کردن داده به کار می رن. Join ها بخشی از دستورات پکیج dplyrr هستن.

دستور مشابه با join در پکیج base با نام merge وجود دارد ولی مزایای join نسبت به merge این است که ترتیب داده ها به هم غریزه، دستور join بصری تر و قابل فهم تره و بر خلاف merge روی DB ها یا حتی SPARK هم قابل استفاده است.

مثل JOIN SQL بر روی فیلدها با مقدار یکسان برقرار می شود که ممکنه لازم باشه از کلیدهای ترکیبی استفاده بشه و همانند قبل به کلید جدول اصلی کلید اصلی و به کلید جدول دیگر کلید فرعی ، ثانویه یا خارجی گفته می شود.

در صورت مشخص نشدن ستونهای کلید، join فیلدهای هم نام را به عنوان کلید در نظر می گیره.

با data.frame و tibble کار می کند.

Mutating joins

یک متغیر از یک مجموعه داده دیگه رو به مجموعه داده فعلی اضافه می کند.

```
inner_join(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ...)
```

```
left_join(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ...)
```

```
right_join(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ...)
```

```
full_join(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ...)
```

اگر اسامی ستونها دقیقاً یکی هستند نیازی به نوشتن اسامی ستونها در دو جدول نیست و تنها یکبار ذکر نام ستون کافی است در غیر این صورت می‌بایست مشخص کنیم کدام ستون ها با هم برابر اند.

به‌عنوان مثال می‌خواهیم داده های bands و artists را به هم متصل نماییم.

```
> bands <- readRDS("C:/Users/Ali Sadeghi Aghili/Dropbox/References/R/Examples/3-  
Cleansing/bands.rds")  
> artists <- readRDS("C:/Users/Ali Sadeghi Aghili/Dropbox/References/R/Examples/3-  
Cleansing/artists.rds")  
  
> bands  
# A tibble: 13 x 3
```

first last band

<chr> <chr> <chr>

John Bonham Led Zeppelin ۱

John Paul Jones Led Zeppelin ۲

Jimmy Page Led Zeppelin ۳

Robert Plant Led Zeppelin ۴

George Harrison The Beatles ۵

John Lennon The Beatles ۶

Paul McCartney The Beatles ۷

Ringo Starr The Beatles ۸

Jimmy Buffett The Coral Reefers ۹

Mick Jagger The Rolling Stones ۱۰

Keith Richards The Rolling Stones ۱۱

Charlie Watts The Rolling Stones ۱۲

Ronnie Wood The Rolling Stones ۱۳

```
> artists  
# A tibble: 16 x 3
```

first last instrument

<chr> <chr> <chr>

Jimmy Buffett Guitar ۱

George Harrison Guitar ۲

Mick Jagger Vocals ۳

Tom Jones Vocals ۴

Davy Jones Vocals ۵

John Lennon Guitar ۶

Paul McCartney Bass ۷

Jimmy Page Guitar ۸

Joe Perry Guitar ۹

Elvis Presley Vocals ۱۰

Keith Richards Guitar ۱۱

Paul Simon Guitar ۱۲

Ringo Starr Drums ۱۳

Joe Walsh Guitar ۱۴

Brian Wilson Vocals ۱۵

Nancy Wilson Vocals ۱۶

همانطور که ملاحظه می‌شود در این جداول ستون های first و last یکسان هستند. بنابراین نیازی به مشخص کردن ستون ها برای اتصال نیست و در زمان اجرا خود R هم به ما پیغامی مبنی بر انتخاب این ستون ها به منظور اتصال می‌دهد.

```
< left_join(x = bands, y = artists)
  Joining, by = c("first", "last")
# A tibble: 13 x 4
```

first last band instrument

```
> chr> <chr> <chr> <chr>
```

John Bonham Led Zeppelin NA ۱

John Paul Jones Led Zeppelin NA ۲

Jimmy Page Led Zeppelin Guitar ۳

Robert Plant Led Zeppelin NA ۴

George Harrison The Beatles Guitar ۵

John Lennon The Beatles Guitar ۶

Paul McCartney The Beatles Bass ۷

Ringo Starr The Beatles Drums ۸

Jimmy Buffett The Coral Reefers Guitar ۹

Mick Jagger The Rolling Stones Vocals ۱۰

Keith Richards The Rolling Stones Guitar ۱۱

Charlie Watts The Rolling Stones NA ۱۲

Ronnie Wood The Rolling Stones NA ۱۳

همچنین برای اطمینان می شد که ستون های اتصال را مشخص نمود.

```
> left_join(x = bands, y = artists, by = c("first", "last"))  
# A tibble: 13 x 4
```

first last band instrument

<chr> <chr> <chr> <chr>

John Bonham Led Zeppelin NA ۱

John Paul Jones Led Zeppelin NA ۲

Jimmy Page Led Zeppelin Guitar ۳

Robert Plant Led Zeppelin NA ۴

George Harrison The Beatles Guitar ۵

John Lennon The Beatles Guitar ۶

Paul McCartney The Beatles Bass ۷

Ringo Starr The Beatles Drums ۸

Jimmy Buffett The Coral Reefers Guitar ۹

Mick Jagger The Rolling Stones Vocals ۱۰

Keith Richards The Rolling Stones Guitar ۱۱

Charlie Watts The Rolling Stones NA ۱۲

Ronnie Wood The Rolling Stones NA ۱۳

در صورت نا هم نام بودن نیست می بایست ستون ها به صورت زیر نوشته می شدند.

```
> left_join(x = bands, y = artists, by = c("first" = "first", "last" = "last"))  
# A tibble: 13 x 4
```

first last band instrument

<chr> <chr> <chr> <chr>

John Bonham Led Zeppelin NA ۱

John Paul Jones Led Zeppelin NA ۲

Jimmy Page Led Zeppelin Guitar ۳

Robert Plant Led Zeppelin NA ۴

George Harrison The Beatles Guitar ۰
John Lennon The Beatles Guitar ۶
Paul McCartney The Beatles Bass ۷
Ringo Starr The Beatles Drums ۸
Jimmy Buffett The Coral Reefers Guitar ۹
Mick Jagger The Rolling Stones Vocals ۱۰
Keith Richards The Rolling Stones Guitar ۱۱
Charlie Watts The Rolling Stones NA ۱۲
Ronnie Wood The Rolling Stones NA ۱۳

اگر چیزی در جدول مرجع باشد که در جدول فرعی نباشد بعد از Join در ستون جدید مقادیر آن برابر NA خواهد بود.

تابع setequal

نام دو مجموعه داده را می‌گیرد و یک logical بر می‌گرداند که آیا مقادیر این دو مجموعه برابرند یا خیر. و در صورت برابر نبودن دلیل نابرابر نبودن دلیل نا برابری هم می‌گوید

```
> set1 <- left_join(x = bands, y = artists)
Joining, by = c("first", "last")
> set2 <- right_join(x = bands, y = artists)
Joining, by = c("first", "last")
> setequal(x = set1, y = set2)
```

FALSE: Different number of rows

تمرین: join ها رو برای دیتاست های albums, stage songs انجام بدهید و بعد با setequal مقایسه‌شون کنید.

```
> albums <- readRDS("C:/Users/Ali Sadeghi Aghili/Dropbox/References/R/Examples/3-
Cleansing/albums.rds")
> stage_songs <- readRDS("C:/Users/Ali Sadeghi Aghili/Dropbox/References/R/Examples/3-
Cleansing/stage_songs.rds")
> set1 <- left_join(x = albums, y = stage_songs)
Joining, by = "year"
> set2 <- right_join(x = albums, y = stage_songs)
Joining, by = "year"
> setequal(x = set1, set2)
```

FALSE: Different number of rows

تمرین: با استفاده از پایپ (% > %) دیتاست bands را به‌عنوان دیتاست مرجع در نظر بگیرید (left join) و با دیتاست join ، artists کنید و بعد اسم، فامیل و گروه گیتاریست ها رو نمایش بدید.

```
> bands %>% left_join(y = artists) %>% filter(instrument == "Guitar") %>% select(first,
                                                                                   last, band)
Joining, by = c("first", "last")
# A tibble: 5 x 3
```

first last band

<chr> <chr> <chr>

Jimmy Page Led Zeppelin ۱

George Harrison The Beatles ۲

John Lennon The Beatles ۲

Jimmy Buffett The Coral Reefers ۴

Keith Richards The Rolling Stones ۵

تمرین منزل: با استفاده از join ها و دیتاستهای artists,songs,bands ، دیتاستی مانند goal بسازید.

```
> goal <- readRDS("C:/Users/Ali Sadeghi Aghili/Dropbox/References/R/Examples/3-
                  Cleansing/goal.rds")
> goal
# A tibble: 3 x 6
```

first last instrument band song album

<chr> <chr> <chr> <chr> <chr> <chr>

Tom Jones Vocals NA It's Not Unusual Along Came Jones ۱

John Lennon Guitar The Beatles Come Together Abbey Road ۲

Paul McCartney Bass The Beatles Hello, Goodbye Magical Mystery Tour ۲

```
> songs <- readRDS("C:/Users/Ali Sadeghi Aghili/Dropbox/References/R/Examples/3-
                  Cleansing/songs.rds")
> songs
# A tibble: 4 x 4
```

song album first last

<chr> <chr> <chr> <chr>

Come Together Abbey Road John Lennon ۱

Dream On Aerosmith STRUEn Tyler ۲

Hello, Goodbye Magical Mystery Tour Paul McCartney ۲

It's Not Unusual Along Came Jones Tom Jones ۴

```
> bands
```

first last band

<chr> <chr> <chr>

John Bonham Led Zeppelin 1

John Paul Jones Led Zeppelin 2

Jimmy Page Led Zeppelin 3

Robert Plant Led Zeppelin 4

George Harrison The Beatles 5

John Lennon The Beatles 6

Paul McCartney The Beatles 7

Ringo Starr The Beatles 8

Jimmy Buffett The Coral Reefers 9

Mick Jagger The Rolling Stones 10

Keith Richards The Rolling Stones 11

Charlie Watts The Rolling Stones 12

Ronnie Wood The Rolling Stones 13

> artists

A tibblele: 16 x 3

first last instrument

<chr> <chr> <chr>

Jimmy Buffett Guitar 1

George Harrison Guitar 2

Mick Jagger Vocals 3

Tom Jones Vocals 4

Davy Jones Vocals 5

John Lennon Guitar 6

Paul McCartney Bass 7

Jimmy Page Guitar 8

Joe Perry Guitar 9

Elvis Presley Vocals 10

Keith Richards Guitar 11

Paul Simon Guitar ۱۲
Ringo Starr Drums ۱۳
Joe Walsh Guitar ۱۴
Brian Wilson Vocals ۱۵
Nancy Wilson Vocals ۱۶

```
> (goal2 <- full_join(x = bands, y = artists) %>% inner_join(y = songs))  
Joining, by = c("first", "last")  
Joining, by = c("first", "last")  
# A tibble: 3 x 6
```

first last band instrument song album

<chr> <chr> <chr> <chr> <chr> <chr>

John Lennon The Beatles Guitar Come Together Abbey Road ۱

Paul McCartney The Beatles Bass Hello, Goodbye Magical Mystery Tour ۲

Tom Jones NA Vocals It's Not Unusual Along Came Jones ۳

```
> setequal(x = goal, y = goal2)
```

TRUE

Filtering joins

در فیلتر کردن داده ها کمک می کند و شامل anti-join, semi-join می باشد. بازهم ۲ جدول می گیرد و براساس دیتای موجود در هر ۲ جدول، یکی را فیلتر می کند.

Semi- join

دیتا را در جدول اول بر اساس جدول دوم فیلتر می کند. مثلاً می خواهیم از دیتای موجود در جدول artists آن هایی را ببینیم که اطلاعات مرتبطی در جدول songs دارند

```
> semi_join(x = artists, y = songs, by = c("first", "last"))  
# A tibble: 3 x 3
```

first last instrument

<chr> <chr> <chr>

Tom Jones Vocals ۱

John Lennon Guitar ۲

Paul McCartney Bass ۳

تمرین: این مثال رو بدون استفاده از semi-join حل کنید.

```
> artists %>% right_join(y = songs, by = c("first", "last")) %>% filter(!is.na(instrument))  
%>% select(first, last, instrument)  
# A tibble: 3 x 3
```

first last instrument

<chr> <chr> <chr>

John Lennon Guitar ۱

Paul McCartney Bass ۲

Tom Jones Vocals ۳

تابع nrow()

این تابع تعداد سطرهای خروجی را به ما می‌دهد.

```
> nrow(bands)  
[1] 13
```

تمرین: با استفاده از semi-join چه تعداد آلبوم توسط بندهای موجود در جدول bands تولید شده اند.

```
> albums %>% semi_join(y = bands) %>% nrow()  
Joining, by = "band"  
[1] 5
```

Anti-join

دیتاهایی که در دیتاست اول هستند ولی دیتای مرتبطی در دیتا ست دوم ندارند را مشخص می‌کند.

```
> artists %>% anti_join(y = bands)  
Joining, by = c("first", "last")  
# A tibble: 8 x 3
```

first last instrument

<chr> <chr> <chr>

Tom Jones Vocals ۱

Davy Jones Vocals ۲

Joe Perry Guitar ۳

Elvis Presley Vocals ۴

Paul Simon Guitar ۵

Joe Walsh Guitar ۶

Brian Wilson Vocals ۷

اپراتورهای مجموعه ها

در کنار join ها در پکیج dplyrr توابع interest, setdiff, union هم وجود دارند که برای تلفیق مجموعه ها می توان از آنها استفاده نمود.

`intersect(x, y, ...)`

`union(x, y, ...)`

`union_all(x, y, ...)`

`setdiff(x, y, ...)`

تابع union

تمام داده های دو مجموعه را بر می گرداند (اگر تکراری وجود داشته باشد یکبار بر می گرداند).

```
> library(babynames)
> set1 <- babynames[1:100, ]
> set2 <- babynames[51:150, ]
> union(x = set1, y = set2)
# A tibble: 150 x 5
```

year sex name n prop

<dbl> <chr> <chr> <int> <dbl>

F Evelyn ۱۲۲ ۰,۰۰۱۲۵ ۱۸۸۰ ۱

F Sara ۱۶۵ ۰,۰۰۱۶۹ ۱۸۸۰ ۲

F Luella ۱۱۱ ۰,۰۰۱۱۴ ۱۸۸۰ ۳

F Lydia ۳۰۲ ۰,۰۰۳۰۹ ۱۸۸۰ ۴

F Elizabeth ۱۹۳۹ ۰,۰۱۹۹ ۱۸۸۰ ۵

F Lola ۱۳۸ ۰,۰۰۱۴۱ ۱۸۸۰ ۶

F Marion ۱۱۵ ۰,۰۰۱۱۸ ۱۸۸۰ ۷

F Jessie ۶۳۵ ۰,۰۰۶۵۱ ۱۸۸۰ ۸

F Flora ۳۶۵ ۰,۰۰۳۷۴ ۱۸۸۰ ۹

F Mae ۳۴۴ ۰,۰۰۳۵۲ ۱۸۸۰ ۱۰

```
# ... with 140 more rows
```

تابع intersect

اشتراک دو مجموعه را بر می گرداند.

```
> intersect(x = set1, y = set2)
# A tibble: 50 x 5
```

year sex name n prop

<dbl> <chr> <chr> <int> <dbl>

F Dora ۵۲۴ ۰,۰۰۵۳۷ ۱۸۸۰ ۱

F Rosa ۵۰۷ ۰,۰۰۵۱۹ ۱۸۸۰ ۲

F Katherine ۵۰۲ ۰,۰۰۵۱۴ ۱۸۸۰ ۳

F Agnes ۴۷۳ ۰,۰۰۴۸۵ ۱۸۸۰ ۴

F Marie ۴۷۱ ۰,۰۰۴۸۳ ۱۸۸۰ ۵

F Nora ۴۷۱ ۰,۰۰۴۸۳ ۱۸۸۰ ۶

F May ۴۶۲ ۰,۰۰۴۷۳ ۱۸۸۰ ۷

F Mamie ۴۳۶ ۰,۰۰۴۴۷ ۱۸۸۰ ۸

F Blanche ۴۲۷ ۰,۰۰۴۳۷ ۱۸۸۰ ۹

F Stella ۴۱۴ ۰,۰۰۴۲۴ ۱۸۸۰ ۱۰

```
# ... with 40 more rows
```

تابع setdiff

داده هایی را بر می گرداند که در مجموعه اولی هستند ولی در مجموعه دوم نیستند.

```
> setdiff(x = set1, y = set2)
# A tibble: 50 x 5
```

year sex name n prop

<dbl> <chr> <chr> <int> <dbl>

F Mary ۷۰۶۵ ۰,۰۷۲۴ ۱۸۸۰ ۱

F Anna ۲۶۰۴ ۰,۰۲۶۷ ۱۸۸۰ ۲

F Emma ۲۰۰۳ ۰,۰۲۰۵ ۱۸۸۰ ۳

F Elizabeth ۱۹۳۹ ۰,۰۱۹۹ ۱۸۸۰ ۴

F Minnie ۱۷۴۶ ۰,۰۱۷۹ ۱۸۸۰ ۵

F Margaret ۱۵۷۸ ۰,۰۱۶۲ ۱۸۸۰ ۶

F Ida ۱۴۷۲ ۰,۰۱۵۱ ۱۸۸۰ ۷

F Alice ۱۴۱۴ ۰,۰۱۴۵ ۱۸۸۰ ۸

F Bertha ۱۳۲۰ ۰,۰۱۳۵ ۱۸۸۰ ۹

F Sarah ۱۲۸۸ ۰,۰۱۳۲ ۱۸۸۰ ۱۰

... with 40 more rows

```
> setdiff(x = set2, y = set1)
```

A tibble: 50 x 5

year sex name n prop

<dbl> <chr> <chr> <int> <dbl>

F Matilda ۲۱۰ ۰,۰۰۲۱۵ ۱۸۸۰ ۱

F Irene ۲۰۴ ۰,۰۰۲۰۹ ۱۸۸۰ ۲

F Kathryn ۲۰۴ ۰,۰۰۲۰۹ ۱۸۸۰ ۳

F Esther ۱۹۸ ۰,۰۰۲۰۳ ۱۸۸۰ ۴

F Willie ۱۹۲ ۰,۰۰۱۹۷ ۱۸۸۰ ۵

F Henrietta ۱۹۱ ۰,۰۰۱۹۶ ۱۸۸۰ ۶

F Ollie ۱۸۳ ۰,۰۰۱۸۷ ۱۸۸۰ ۷

F Amy ۱۶۷ ۰,۰۰۱۷۱ ۱۸۸۰ ۸

F Rachel ۱۶۶ ۰,۰۰۱۷۰ ۱۸۸۰ ۹

F Sara ۱۶۵ ۰,۰۰۱۶۹ ۱۸۸۰ ۱۰

... with 40 more rows

dplyrr در bind

همانند bind در پکیج base عمل می‌کند و شامل bind-rows و bind-cols است.

دقت کنید که در bind کردن مخصوصاً bind-cols فرض بر این است که ترتیب قرارگیری مقادیر در دو دیتاست ترتیب مورد نظر برای تلفیق هست. اگر نه دیتا بهم می‌ریزد.

فرقشون نسبت به package base این هست که هم سریع تر هستن، هم tibble بر می‌گردونن و هم یک امکانی تحت عنوان id دارن.

Bind-cols

ستونها را به هم می چسباند.

```
> df1 <- mtcars[, 5:dim(mtcars)[2]]  
> df2 <- mtcars[, 1:4]  
> bind_cols(df1, arrange(df2, desc(mpg)))
```

drat wt qsec vs am gear carb mpg cyl disp hp

۶۵	۷۱,۱	۴	۳۳,۹	۴	۴	۱	۱۶,۴۶	۲,۶۲۰	۳,۹۰	۱
۶۶	۷۸,۷	۴	۳۲,۴	۴	۴	۱	۱۷,۰۲	۲,۸۷۵	۳,۹۰	۲
۵۲	۷۵,۷	۴	۳۰,۴	۱	۴	۱	۱۸,۶۱	۲,۳۲۰	۳,۸۵	۳
۱۱۳	۹۵,۱	۴	۳۰,۴	۱	۳	۰	۱۹,۴۴	۳,۲۱۵	۳,۰۸	۴
۶۶	۷۹,۰	۴	۲۷,۳	۲	۳	۰	۱۷,۰۲	۳,۴۴۰	۳,۱۵	۵
۹۱	۱۲۰,۳	۴	۳۶,۰	۱	۳	۰	۲۰,۲۲	۳,۴۶۰	۲,۷۶	۶
۶۲	۱۴۶,۷	۴	۲۴,۴	۴	۳	۰	۱۵,۸۴	۳,۵۷۰	۳,۲۱	۷
۹۳	۱۰۸,۰	۴	۲۲,۸	۲	۴	۰	۲۰,۰۰	۳,۱۹۰	۳,۶۹	۸
۹۵	۱۴۰,۸	۴	۲۲,۸	۲	۴	۰	۲۲,۹۰	۳,۱۵۰	۳,۹۲	۹
۹۷	۱۲۰,۱	۴	۲۱,۵	۴	۴	۰	۱۸,۳۰	۳,۴۴۰	۳,۹۲	۱۰
۱۱۰	۲۵۸,۰	۶	۲۱,۴	۴	۴	۰	۱۸,۹۰	۳,۴۴۰	۳,۹۲	۱۱
۱۰۹	۱۲۱,۰	۴	۲۱,۴	۳	۳	۰	۱۷,۴۰	۴,۰۷۰	۳,۰۷	۱۲
۱۱۰	۱۶۰,۰	۶	۲۱,۰	۳	۳	۰	۱۷,۶۰	۳,۷۳۰	۳,۰۷	۱۳
۱۱۰	۱۶۰,۰	۶	۲۱,۰	۳	۳	۰	۱۸,۰۰	۳,۷۸۰	۳,۰۷	۱۴
۱۷۵	۱۴۵,۰	۶	۱۹,۷	۴	۳	۰	۱۷,۹۸	۵,۲۵۰	۳,۹۳	۱۵
۱۲۳	۱۶۷,۶	۶	۱۹,۲	۴	۳	۰	۱۷,۸۲	۵,۴۲۴	۳,۰۰	۱۶
۱۷۵	۴۰۰,۰	۸	۱۹,۲	۴	۳	۰	۱۷,۴۲	۵,۳۴۵	۳,۲۳	۱۷
۱۷۵	۳۶۰,۰	۸	۱۸,۷	۱	۴	۱	۱۹,۴۷	۲,۲۰۰	۴,۰۸	۱۸
۱۰۵	۲۲۵,۰	۶	۱۸,۱	۲	۴	۱	۱۸,۵۲	۱,۶۱۵	۴,۹۳	۱۹
۱۲۳	۱۶۷,۶	۶	۱۷,۸	۱	۴	۱	۱۹,۹۰	۱,۸۳۵	۴,۲۲	۲۰
۱۸۰	۲۷۵,۸	۸	۱۷,۳	۱	۳	۰	۲۰,۰۱	۲,۴۶۵	۳,۷۰	۲۱
۱۸۰	۲۷۵,۸	۸	۱۶,۴	۲	۳	۰	۱۶,۸۷	۳,۵۲۰	۳,۷۶	۲۲
۲۶۴	۳۵۱,۰	۸	۱۵,۸	۲	۳	۰	۱۷,۳۰	۳,۴۳۵	۳,۱۵	۲۳
۱۵۰	۳۱۸,۰	۸	۱۵,۵	۴	۳	۰	۱۵,۴۱	۳,۸۴۰	۳,۷۳	۲۴
۱۸۰	۲۷۵,۸	۸	۱۵,۲	۲	۳	۰	۱۷,۰۵	۳,۸۴۵	۳,۰۸	۲۵
۱۵۰	۳۰۴,۰	۸	۱۵,۲	۱	۴	۱	۱۸,۹۰	۱,۹۳۵	۴,۰۸	۲۶

۳۳۵ ۳۰۱,۰ ۸ ۱۵,۰ ۲۵ ۱۰ ۱۶,۷۰ ۳,۱۴۰ ۴,۴۳ ۳۷
 ۲۳۰ ۴۴۰,۰ ۸ ۱۴,۷ ۲۵ ۱۱ ۱۶,۹۰ ۱,۵۱۳ ۳,۷۷ ۲۸
 ۲۴۵ ۳۶۰,۰ ۸ ۱۴,۳ ۴۵ ۱۰ ۱۴,۵۰ ۳,۱۷۰ ۴,۲۲ ۳۹
 ۲۴۵ ۳۵۰,۰ ۸ ۱۳,۳ ۶۵ ۱۰ ۱۵,۵۰ ۲,۷۷۰ ۳,۶۲ ۳۰
 ۲۰۵ ۴۷۲,۰ ۸ ۱۰,۴ ۸۵ ۱۰ ۱۴,۶۰ ۳,۵۷۰ ۳,۵۴ ۳۱
 ۲۱۵ ۴۶۰,۰ ۸ ۱۰,۴ ۲۴ ۱۱ ۱۸,۶۰ ۲,۷۸۰ ۴,۱۱ ۳۲

Bind-rows

سطرها را با هم تلفیق می‌کند به شرطی که تعداد ستونها برابر باشند. در عین حال یک آرگومان بسیار کاربردی. id را دارد که از آن طریق می‌تواند داده‌های هر دسته را مشخص کند.

```
> ali <- data_frame(point = 18, year = 1396, major = "Automation")
> babak <- data_frame(point = 19, year = 1396, major = "Programming")
> bind_rows(ali, babak)
# A tibble: 2 x 3
```

point year major

<dbl> <dbl> <chr>

Automation ۱۳۹۶ ۱۸ ۱

Programming ۱۳۹۶ ۱۹ ۲

```
> bind_rows(ali, babak, .id = "id")
# A tibble: 2 x 4
```

id point year major

<chr> <dbl> <dbl> <chr>

Automation ۱۳۹۶ ۱۸ ۱ ۱

Programming ۱۳۹۶ ۱۹ ۲ ۲

```
> list <- list(ali = ali, babak = babak)
> list
```

`ali`\$

```
# A tibble: 1 x 3
```

point year major

<dbl> <dbl> <chr>

Automation ۱۳۹۶ ۱۸ ۱

babak\$

```
# A tibble: 1 x 3
```

point year major

<dbl> <dbl> <chr>

Programming ۱۳۹۶ ۱۹ ۱

```
> bind_rows(list,.id = "name")
# A tibble: 2 x 4
```

name point year major

<chr> <dbl> <dbl> <chr>

ali ۱۸ ۱۳۹۶ Automation ۱

babak ۱۹ ۱۳۹۶ Programming ۲

تمرین: با استفاده از دیتاست های discography و jimi یک data.frame از تمام داده ها به همراه داده های سال های مربوطه بسازید.

```
> jimi %>% bind_rows(.id = "album") %>% left_join(y = discography)
Joining, by = "album"
# A tibble: 39 x 4
```

album song length year

<chr> <chr> <time> <int>

Are You Experienced Purple Haze " ۹۹۶۰ secs" ۱۹۶۷ ۱

Are You Experienced Manic Depression ۱۳۵۶۰ secs ۱۹۶۷ ۲

Are You Experienced Hey Joe ۱۲۱۸۰ secs ۱۹۶۷ ۳

Are You Experienced May This Be Love ۱۱۶۴۰ secs ۱۹۶۷ ۴

Are You Experienced I Don't Live Today ۱۴۱۰۰ secs ۱۹۶۷ ۵

Are You Experienced The Wind Cries Mary ۱۲۰۶۰ secs ۱۹۶۷ ۶

Are You Experienced Fire " ۹۲۴۰ secs" ۱۹۶۷ ۷

Are You Experienced Third Stone from the Sun ۲۴۰۰۰ secs ۱۹۶۷ ۸

Are You Experienced Foxy Lady ۱۱۷۰۰ secs ۱۹۶۷ ۹

Are You Experienced Are You Experienced? ۱۴۱۰۰ secs ۱۹۶۷ ۱۰

```
# ... with 29 more rows
```

data_frame

data. Frame است ولی string ها را به factor تبدیل نمی‌کند، اسم ناخواسته به سطرها نمی‌دهد، اسامی ستونها را تغییر نمی‌دهد و recycle نمی‌کند.

همچنین امکان ایجاد lazy Argamet (آرگومانی که از آرگومانهای قبلی خودش استفاده کند) را به ما داده و خروجی اش tibblele است.

```
> name <- c("Ali", "Babak", "Parviz")
> family <- c("Sadeghi Aghili", "Pirooz", "Agha Sadeghi")
> age <- c(30, 35, 50)
> major <- c("DS", "BI", "DB")
> df1 <- data.frame(name, family, age, major)
> glimpse(df1)
```

Observations: ۳

Variables: ۴

name <fct> Ali, Babak, Parviz \$

family <fct> Sadeghi Aghili, Pirooz, Agha Sadeghi \$

age <dbl> ۳۰, ۳۵, ۵۰ \$

major <fct> DS, BI, DB \$

```
> df2 <- data_frame(name, family, age, major)
> glimpse(df2)
```

Observations: ۳

Variables: ۴

"name <chr> "Ali", "Babak", "Parviz \$

"family <chr> "Sadeghi Aghili", "Pirooz", "Agha Sadeghi \$

age <dbl> ۳۰, ۳۵, ۵۰ \$

"major <chr> "DS", "BI", "DB \$

```
> data_frame(nums = 1:5, nums2 = nums^2)
# A tibblele: 5 x 2
```

nums nums۲

<int> <dbl>

۱ ۱ ۱

۴ ۲ ۲

۹ ۳ ۳

۱۶۴۴

۲۵۵۵

```
> data.frame(nums = 1:5, nums2 = nums^2)
Error in data.frame(nums = 1:5, nums2 = nums^2) : object 'nums' not found
```

Coercion rules

R در نهایت می‌بایست مقادیر هر بردار رو به صورت تک جنسی ذخیره کند. یعنی هم عناصر از یک جنس باشند. R به این جنسها atomic vector می‌گه عبارتند از

```
> #logical
> typeof(TRUE)
[1] "logical"
```

```
> #characterer
> typeof("Hello")
[1] "characterer"
```

```
> #double
> typeof(3.14)
[1] "double"
```

```
> #integer
> typeof(3L)
[1] "integer"
```

```
> #complex
> typeof(1 + 2i)
[1] "complex"
```

```
> #raw
> typeof(raw(1))
[1] "raw"
```

مشخص می‌شن () typeof این جنسها با

دقت کنید که typeof از class متفاوت است. Class جنس شی رو به ما می‌گوید و typeof جنس محتوای شی رو. R اگر با انواع مختلف داده توی یک بردار روبرو بشه مجبوره همرو به یکی از این atomic ها برگردونه.

قواعدی که R باهاشون تصمیم میگیره که هرم بردار ترکیبی رو در نهایت با چه جنسی ذخیره کنه Coercion Rules یا قواعد اجباری کن.

ولی در مورد factor ها موضوع پیچیده تره. تبدیل فاکتور به عدد یا کاراکتر باید دقت کرد که فاکتور ها یک عدد دارند و یک لیبل دارند که به ان مرتبط است.

```
> fac <- as.factor(c("A", "B", "C", "B"))
> fac
[1] A B C B
```

Levels: A B C

```
> as.characterer(fac)
[1] "A" "B" "C" "B"
> as.numeric(fac)
[1] 1 2 3 2
```

در مورد فاکتور های رشته ای مشکلی وجود ندارد ولی فاکتور های عددی می توانند موجب سردرگمی شوند.

```
> fac2 <- as.factor(c(4, 5, 6, 5))
> fac2
[1] 4 5 6 5
```

Levels: ۴ ۵ ۶

```
> as.characterer(fac2)
[1] "4" "5" "6" "5"
> as.numeric(fac2)
[1] 1 2 3 2
```

برای جلوگیری از بروز این مشکل می بایست ابتدا فاکتور های عددی حتما به رشته و سپس به عدد تبدیل شوند.

```
> as.numeric(as.characterer(fac2))
[1] 4 5 6 5
```

تمرین: دیتاست های seventies و eighties را با هم bind قبل از bird، جنس ستونها رو چک کنید و ببینید که چه اتفاقی می افتد.

```
> seventies
# A tibble: 10 x 3
```

year album band

<fct> <chr> <chr>

Bridge Over Troubled Water Simon and Garfunkel ۱۹۷۰ ۱

Jesus Christ Superstar Various Artists ۱۹۷۱ ۲

Harvest Neil Young 1972 2
The World is a Ghetto War 1973 2
Goodbye Yellow Brick Road Elton John 1974 0
Elton John's Greatest Hits Elton John 1970 1
Peter Frampton Frampton Comes Alive 1976 7
Rumours Fleetwood Mac 1977 8
Saturday Night Fever Bee Gees 1978 9
Billy Joel 92nd Street 1979 10

```
> eighties  
# A tibble: 10 x 3
```

year album band

<chr> <chr> <chr>

The Wall Pink Floyd 1980 1
Hi Infidelity REO Speedwagon 1981 2
Asia Asia 1982 3
Thriller Michael Jackson 1982 2
Thriller Michael Jackson 1982 0
Born in the U.S.A. Bruce Springsteen 1980 1
Whitney Houston Whitney Houston 1987 7
Slippery When Wet Bon Jovi 1987 8
Faith George Michael 1988 9
Don't Be Cruel Bobby Brown 1989 10

```
> bind_rows(seventies, eighties)  
# A tibble: 20 x 3
```

year album band

<chr> <chr> <chr>

Bridge Over Troubled Water Simon and Garfunkel 1970 1
Jesus Christ Superstar Various Artists 1971 2
Harvest Neil Young 1972 3
The World is a Ghetto War 1973 2
Goodbye Yellow Brick Road Elton John 1974 0
Elton John's Greatest Hits Elton John 1970 1

Peter Frampton Frampton Comes Alive ۱۹۷۶ ۷

Rumours Fleetwood Mac ۱۹۷۷ ۸

Saturday Night Fever Bee Gees ۱۹۷۸ ۹

Billy Joel ۵۲nd Street ۱۹۷۹ ۱۰

The Wall Pink Floyd ۱۹۸۰ ۱۱

Hi Infidelity REO Speedwagon ۱۹۸۱ ۱۲

Asia Asia ۱۹۸۲ ۱۳

Thriller Michael Jackson ۱۹۸۳ ۱۴

Thriller Michael Jackson ۱۹۸۴ ۱۵

Born in the U.S.A. Bruce Springsteen ۱۹۸۵ ۱۶

Whitney Houston Whitney Houston ۱۹۸۶ ۱۷

Slippery When Wet Bon Jovi ۱۹۸۷ ۱۸

Faith George Michael ۱۹۸۸ ۱۹

Don't Be Cruel Bobby Brown ۱۹۸۹ ۲۰

```
Warning messages:
1: In bind_rows(x, .id) :
```

binding factor and characterer vector, coercing into characterer vector

```
2: In bind_rows(x, .id) :
```

binding characterer and factor vector, coercing into characterer vector

تمرین: دیتاست های *sixties* و *sevent* را با هم *bind* کنید.

```
> sixties
# A tibble: 10 x 3
```

year album band

<int> <chr> <chr>

The Sound of Music Original Broadway Cast ۱۹۶۰ ۱

Camelot Original Broadway Cast ۱۹۶۱ ۲

West Side Story Soundtrack ۱۹۶۲ ۳

West Side Story Soundtrack ۱۹۶۳ ۴

Hello, Dolly! Original Broadway Cast ۱۹۶۴ ۵

Mary Poppins Soundtrack ۱۹۶۵ ۶

Whipped Cream & Other Delights Herb Alpert & The Tijuana Brass ۱۹۶۶ ۷

More of The Monkees The Monkees ۱۹۶۷ ۸

Are You Experienced? The Jimi Hendrix Experience 1968 9

In-A-Gadda-Da-Vida Iron Butterfly 1969 10

```
> seventies
# A tibble: 10 x 3
```

year album band

<fct> <chr> <chr>

Bridge Over Troubled Water Simon and Garfunkel 1970 1

Jesus Christ Superstar Various Artists 1971 2

Harvest Neil Young 1972 3

The World is a Ghetto War 1973 4

Goodbye Yellow Brick Road Elton John 1974 5

Elton John's Greatest Hits Elton John 1975 6

Peter Frampton Frampton Comes Alive 1976 7

Rumours Fleetwood Mac 1977 8

Saturday Night Fever Bee Gees 1978 9

Billy Joel 52nd Street 1979 10

```
> bind_rows(sixties, seventies)
Error in bind_rows_(x, .id) :
```

Column `year` can't be converted from integer to factor

```
> seventies %>% mutate(year = as.numeric(as.character(year))) %>% bind_rows(sixties)
# A tibble: 20 x 3
```

year album band

<dbl> <chr> <chr>

Bridge Over Troubled Water Simon and Garfunkel 1970 1

Jesus Christ Superstar Various Artists 1971 2

Harvest Neil Young 1972 3

The World is a Ghetto War 1973 4

Goodbye Yellow Brick Road Elton John 1974 5

Elton John's Greatest Hits Elton John 1975 6

Peter Frampton Frampton Comes Alive 1976 7

Rumours Fleetwood Mac ۱۹۷۷ ۸
 Saturday Night Fever Bee Gees ۱۹۷۸ ۹
 Billy Joel ۵۲nd Street ۱۹۷۹ ۱۰
 The Sound of Music Original Broadway Cast ۱۹۶۰ ۱۱
 Camelot Original Broadway Cast ۱۹۶۱ ۱۲
 West Side Story Soundtrack ۱۹۶۲ ۱۳
 West Side Story Soundtrack ۱۹۶۳ ۱۴
 Hello, Dolly! Original Broadway Cast ۱۹۶۴ ۱۵
 Mary Poppins Soundtrack ۱۹۶۵ ۱۶
 Whipped Cream & Other Delights Herb Alpert & The Tijuana Brass ۱۹۶۶ ۱۷
 More of The Monkees The Monkees ۱۹۶۷ ۱۸
 Are You Experienced? The Jimi Hendrix Experience ۱۹۶۸ ۱۹
 In-A-Gadda-Da-Vida Iron Butterfly ۱۹۶۹ ۲۰

تابع rownames- to- colam

از توابع پکیج tibblele هست که به ما کمک می‌کند به راحتی عناوین سطرها را در قالب یک ستون به dataset اضافه کنیم .

```
> rownames_to_column(df = mtcars[1:3,])
```

rowname mpg cyl disp hp drat wt qsec vs am gear carb

Mazda RX۴ ۲۱,۰ ۶ ۱۶۰ ۱۱۰ ۳,۹۰ ۲,۶۲۰ ۱۶,۴۶۰ ۱ ۴ ۴ ۱

Mazda RX۴ Wag ۲۱,۰ ۶ ۱۶۰ ۱۱۰ ۳,۹۰ ۲,۸۷۵ ۱۷,۰۲۰ ۱ ۴ ۴ ۳

Datsun ۷۱۰ ۲۲,۸ ۴ ۱۰۸ ۹۳ ۳,۸۵ ۲,۳۲۰ ۱۸,۶۱ ۱ ۱ ۴ ۱ ۳

```
> rownames_to_column(df = mtcars[1:3,], var = "carname")
```

carname mpg cyl disp hp drat wt qsec vs am gear carb

Mazda RX۴ ۲۱,۰ ۶ ۱۶۰ ۱۱۰ ۳,۹۰ ۲,۶۲۰ ۱۶,۴۶۰ ۱ ۴ ۴ ۱

Mazda RX۴ Wag ۲۱,۰ ۶ ۱۶۰ ۱۱۰ ۳,۹۰ ۲,۸۷۵ ۱۷,۰۲۰ ۱ ۴ ۴ ۳

Datsun ۷۱۰ ۲۲,۸ ۴ ۱۰۸ ۹۳ ۳,۸۵ ۲,۳۲۰ ۱۸,۶۱ ۱ ۱ ۴ ۱ ۳

تابع has-rowamas

از توابع پکیج tibblele هست که تنها مشخص می‌کند که دیتاست label دارد یا خیر.

```
> has_rownames(mtcars)
```

مشکلاتی که در join ها به وجود می آیند.

مشکلاتی که در زمان استفاده از join ها ممکن است با آن ها مواجه شویم عبارتند از

کلید گم شده

ستون گمشده کلید

ستون غیر کلید تکراری

کلید تکراری

> shows

A tibble: 6 x 2

musical year

<chr> <int>

The Sound of Music ۱۹۵۹ ۱

Into the Woods ۱۹۸۶ ۲

The King and I ۱۹۵۱ ۳

West Side Story ۱۹۵۷ ۴

Cats ۱۹۸۱ ۵

Phantom of the Opera ۱۹۸۶ ۶

> composers

A tibble: 8 x 2

musical composer

<chr> <chr>

Cats Andrew Lloyd Webber ۱

Into the Woods Stephen Sondheim ۲

Phantom of the Opera Andrew Lloyd Webber ۳

The King and I Richard Rogers ۴

The King and I Oscar Hammerstein II ۵

The Sound of Music Richard Rogers ۶

The Sound of Music Oscar Hammerstein II ۷

West Side Story Leonard Bernstein ۸

> shows %>% left_join(composers)

```
Joining, by = "musical"  
# A tibble: 8 x 3
```

musical year composer

<chr> <int> <chr>

The Sound of Music ۱۹۵۹ Richard Rogers ۱

The Sound of Music ۱۹۵۹ Oscar Hammerstein II ۲

Into the Woods ۱۹۸۶ Stephen Sondheim ۳

The King and I ۱۹۵۱ Richard Rogers ۴

The King and I ۱۹۵۱ Oscar Hammerstein II ۵

West Side Story ۱۹۵۷ Leonard Bernstein ۶

Cats ۱۹۸۱ Andrew Lloyd Webber ۷

Phantom of the Opera ۱۹۸۶ Andrew Lloyd Webber ۸

کلید تکراری

dplyr با کلید تکراری مشکلی ندارد و هر دو رو در نظر می‌گیره که این ممکنه data set رو زیادی بزرگ کنه.

تمرین: دیتاست های ستون کلید گم شده stage-writers, stage_songs را join کنید.

کلید گم شده کلید مربوط در ستون کلیدهای اصلی وجود ندارد.

ستون غیر کلید تکراری

در صورتی که ستونهای غیر کلید تکراری در dataset ها وجود داشته باشند dplyr به صورت اتوماتیک به آنها پیوندی دهد مگر آنکه پسوند به صورت دستی مشخص شود.

```
> inner_join(work, study)  
Joining, by = c("name", "major")  
# A tibble: 0 x 2  
# ... with 2 variables: name <chr>, major <chr>  
> inner_join(work, study, by = "name")  
# A tibble: 3 x 3
```

name major.x major.y

<chr> <chr> <chr>

ali Automation & Advanced Analytics Automation ۱

babak Business Intelligence & Data Visualization Software Engineering ۲

parviz Database Administration Chemical Engineering ۲

```
> inner_join(work, study, by = "name", suffix= c("work", "study"))  
# A tibble: 3 x 3
```

name majorwork majorstudy

<chr> <chr> <chr>

ali Automation & Advanced Analytics Automation ۱

babak Business Intelligence & Data Visualization Software Engineering ۲

parviz Database Administration Chemical Engineering ۲

می‌شود به جای استفاده از suffix از suffin از تابع rename استفاده کرد.

تمرین: داده های elvis-movie, elvis-sang رو با هم join کنید و بعد اسم ستون name رو به name.y movie به sany تمیز بدهید

Join کردن بیش از ۲ dataset

برای اینکار می‌بایست به ترتیب data set ها رو با هم join کرد. ولی می‌توان از تابع reduce کمک گرفت.

Reduce

تابع reduce یکی از توابع پکیج purrr هست و به داده های dataset ها رو بر اساس تابع معرفی شده و آرگومانهای مربوط با هم تلفیق می‌کنه ولی برای اینکار اول باید به نحوی دیتاست ها رو از هم در یک محل متمایز و در کنار هم ذخیره کنیم. که بهترین روش برای اینکار ذخیره اونها در یک list هست.

تابع merge

یکی از توابع package base هست که صورت کار join رو می‌کند. برای full join ، آرگومان all=TRUE و برای inner join قرار می‌گیرد.

تاریخ و زمان در R

اصولاً نگارش تاریخ در کشورهای مختلف می‌تواند به اشکال مختلفی صورت پذیرد ولی یک استاندارد جهانی برای اینکار وجود دارد که می‌گوید اجزای تاریخ می‌بایست از بزرگ به کوچک نوشته شوند (ISO ۸۶۰۱) و در

عین حال تعداد کارکترهای هر بخش مشخص هستند. (سال ۴ تا و ماه هر کدام ۲ کاراکتر) همینطور جدا کننده در این استاندارد فاصله یا خط تیره است.

یادآوری تابع `as.Date` یک رشته می پذیرد که می بایست با فرمت استاندارد باشد یا فرمت آن مشخص شود. یادآوری برخی پکیجها از جمله `readr` در موقع ورود داده معمولاً فرمت تاریخ را تشخیص می دهند. خصوصاً اگر به فرمتهای معمول باشد.

پکیج `anytime`

این پکیج و تابع اصلی آن تابع `any time` () برای راحتی تبدیل واستفاده از زمان و تاریخ بدون نیاز به تغییر فرمت ایجاد شده است و فرمتهای بسیار زیادی را پشتیبانی می کند.

یادآوری

تاریخ در R از ۱۹۷۰/۱/۱ به صورت عدد ذخیره می شود. پس می توان روی آنها اعمال ریاضی انجام داد یا آنها را کشید.

پکیج `lubridate`

یکی از اعضای خانواده tidyverse هست که کمک می کند تا به راحتی هر چه تمام تر بتوان به تاریخ و زمان کارکرد. یکی از مهمترین مزایای این پکیج این هست که برایش مهم نیست با فرمت تاریخ کار داره یا زمان و به راحتی کارها رو انجام میده. همینطور به راحتی می شود بخشهای زمان رو ازش جدا کرد یا یک بازه رتا بهش فهموند.

توابع تبدیل زمان

شامل

اگر `time fone` مشخص نشود، `lubridate` ، آنرا `utc` فرض می کند.

`Lubridate` نزدیک به `timelow boo` را می شناسد.

تابع `parse_date_time`

نسبت به توابع ۶۳ حرفی تبدیل تاریخ و زمان کامل تر است ولی مهمترین مزیت آن این است که می توان چندین فرمت خروجی را به صورت همزمان برای چندین ورودی اعمال کرد.

Make-dateine, make-datece

با استفاده از این تابع می‌توان اجزای مجزای محدودی را با یکدیگر ادغام نموده و به صورت زمان تاریخ تبدیل نمود.

تمرین: داده های دیتاست `aki-weather-hourlyg-۲۰۱۶-csv` را خوانده، سپس با استفاده از تابع `makedate` و توابعی که پیش از این آموخته اید، یک فیلد `date` و یک فیلد `datetime` بسازید.

تمرین: در دیتای مربوط به `releasetime` بگویید فراوانی `relase` ها در هر سال چقدر بوده و بگویید چه کسری از `release` ها در صبح اتفاق افتاده. اجزای هر `date` یا `datetime` رو هم می‌شود با استفاده از تابع مربوطه تغییر داد.

توابع `lubridate` در `rownd`

در `lubridate` سه تابع برای `round` کردن زمان وجود دارند. ----

هر سه یک آرگومان `umit` دارند که میزان رند شدن را مشخص می‌کند.

آرگومان `umit` می‌تواند شامل یک عدد و `umit` باشد مثل `zgeas`

تمرین: اختلاف زمان امروز از ابتدای هفته را بدست آورید.

تمرین: با استفاده از دیتاست `aki-haurly` بگویید در هر ساعت چند مشاهده انجام شده است ؟

راهنمایی: برای اینکار ابتدایی بایست از طریق رند کردن یک ستون برای هر ساعت در هر روز ایجاد کرد(تا ساعت رند کرد)

تابع difftime

با استفاده از این تابع می‌توان اختلاف بین دو زمان را بر حسب واحد ارائه شده بدست آورد.

تمرین: اختلاف امروز و ۲ روز پیش رو بر حسب ساعت می‌خواهیم.

بازه های زمانی

حتی زمانی که ما می‌گوییم ۲ روز را در تاریخ کم کن نمی‌توان مطمئن بود که واقعاً ۴۸ ساعت از تاریخ فوق کم شده یا خیر. پس دقیقاً باید تبدیل کنیم که منظورمون چی هست؟ برای اینکار lubridate از ۲ تابع duration, period استفاده می‌کند.

تابع period

همون مفهوم انسانی زمان رو پیاده می‌کند. مثلاً یک زمان منهای یک پرپود یک روزه زمانی روز قبل و همون زمان رو می‌ده حتی اگر این اختلاف کمتر از ۲۴ ساعت باشه.

تابع Duration

این تابع به شکل دقیق و کامپیوتری و بر حسب تعداد ثانیه های تشکیل دهنده هر ساعت، زمانها را محاسبه می‌کند. یعنی اگر بگوییم از این ساعت در امروز یک روز برو به عقب امروز را منهای ۸۶/۴۰۰ ثانیه می‌کند. Period ها با توابع به شکل عادی و duration ها با توابعی با یک d اضافه اولشون شروع میشن.

تمرین: می‌خواهیم هفته دیگر همین ساعت یک ایمیل بفرستیم. هفته دیگه همین ساعت رو بدست بیاورید.

تمرین: می‌خواهیم که تا پس فردا یک کاری انجام شده باشه. پس فردا بدست بیارید.

تمرین: می‌خواهیم از تاریخ امروز به مدت ۱۲ هفته هر هفته بیایم کلاس. تاریخ کلاس ها رو بدست بیارین.

شکل نابرابری ماه ها

فرض کنید می‌خواهیم به ۳۱ ژانویه ماه به ماه اضافه کنیم تا آخر هر ماه رو بدست بیاریم.

بسیاری از مقادیر NA بر می‌گردند چون تمام ماه ها ۳۱ روز ندارند. برای حل این مشکل از در اپراتور ----- به ترتیب برای جمع و تفریق استفاده می‌شود.

Interval یا بازه زمانی

بر خلاف period ها، بازه های زمانی نقطه شروع و پایان مشخصی دارند. و با تابع interval یا اپراتور %--% مشخص می شوند.

Int-end,int-start

یک بازه می گیرند و ابتدا و انتهای آنرا بر می گردانند.

Int-lenget

طول بازه را به صورت Dwratio بر می گردانند.

طول بازه را به صورت as.rurdtion هم همین کار را می کند.

as.period تابع

طول بازه را به صورت periced بر می گردانند.

%within% اپراتور

برای بررسی اینکه یک مقطع زمانی در بازه زمانی خاصی وجود دارد یا خیر به کار می رود.

تابع int-ouelupe

دو بازه می گیرد و می گوید آیا این دو بازه هم پوشانی دارند یا خیر ---

جمع بندی

اگر نقطه شروع و پایان مشخصی وجود دارد. از intervals استفاده کنید.

اگر قراره از نتیجه کار انسان استفاده کند معمولاً باید از period استفاده کنید.

اگر دقیقاً فواصل زمانی براتون مهمه از Duration استفاده کنید.

تمرین: با استفاده از داده های monarchs که مربوط به پادشاهی های England هست، گزارش ارائه دهید که در آن اسامی پادشاهان، طول سلطنت، نمایش داده شود و بر اساس طول سلطنت به ترتیب نزولی مرتب شده باشد.

راهنمایی می بایست ابتدا بازه ای برای مدت سلطنت ایجاد شود .

محدوده‌های زمانی (time towe) منطقه زمانی

برای مشخص کردن مکان زمانی در هر نقطه جغرافیایی به کار می رود .

Sys.timezone

مکان زمانی کامپیوتر را به ما می‌دهد.

OlsonNames

تمامی مکان های زمانی موجود را به ما می‌دهد. و از طریق یک مرجع جهانی update می شه. همون مرجعی که متولی IP دهی هم هست.

Timezone دستکاری

دو صورت ساله ممکن است. می‌خواهیم دقیقاً زمان معادل در یک منطقه را برای منطقه ای دیگر ببایم یا می‌خواهیم زمان عددی ثابتی را در تمام مناطق مختلف به کار می رود.

تابع with_tz

برای بدست آوردن زمان مشخص یکسان در بازه زمانی دیگر به کار می رود.

پردازش سریع زمان و تاریخ

با توجه به آنکه تابع parse-date-time می‌بایست format ها را چک بکند ممکن است گاهی کند باشد. برای اینکار پکیج sttime و تابع fast_strptime () وجود دارند.

تابع fast-strptime

در این تابع می‌بایست تاریخ و فرمت دقیق تاریخ همانند آنچه در as.Date مشخص می شد. مشخص شود.

تابع stamp

یک تاریخ یا زمان می‌گیرد و از طریق آن تابعی می سازد که با استفاده از ان می‌توان تمام تاریخ ها را به یک فرمت دلخواه در آورد .

بیشتر برای گزارش گیری کاربرد دارد.

پکیج data.table

برای کار با داده های حجیم استفاده شود.

Data.table

در واقع همان data.frame هست که تهیه شده برای زمان پردازش کوتاه تر و برنامه نویسی کمتر می گوید بر اساس شروط ما، دیتاست رو فیلتر کن و بعد روی هر گروه که توسط bg مشخص شده عملیات ز رو انجام بده. Data.table به لحاظ عملکردی بهتر است که اعداد صحیح با استفاده از ا مشخص شوند.

برای فیلتر کردن داده به کار می رود

از انجایی که " و " در datatables معنایی متفاوت از dataframe دارد گذاشتن همیشگی آن اجباری نیست. تمرین: یک دیتا table مانند datatable قبل بسازید. عنصر یکی مانده به آخر آنرا نمایش دهید و سپس از روی دیتاست اصلی یک دیتاست بسازید که در آن یکبار خط دوم و دوبار خط سوم تکرار شده باشد.

عملگر N.

تعداد سطرهای هر گروه را می شمارد

عملگر Z

ستونها و اعمال مربوط به آنها را انتخاب می کند.

اپراتور ()

از آنجا که هر ستون به تنهایی به صورت بردار بر می گردد برای انتخاب همزمان چند ستون می بایست آنها را به صورت list در آورد که در data table برای راحتی کار می توان از ()، استفاده نمود و ستونها را درون آن قرار داد. در واقع () به معنی همان لیست است.

در صورت هم اندازه نبودن اندازه بردارها، بردار کوتاه تر (recycle) می شود.

خروجی DT[,B], DT[,.(B)] با هم تفاوت خواهند داشت و استفاده از ()، نتیجه را به صورت DT برگرداند.

تمرین: از دیتاست DT ، و با استفاده از داده های سطرها ۲ تا ۵، ستون B و یک ستون جدید حاصل از ضرب A در برابر C را نمایش دهید.

Bg

به وسیله این آرگومان می توان نتایج را گروه بندی کرد.

ستونهای by در ابتدا نمایش داده می شوند.

Bg می تواند فرمول هم بگیرد

اگر تنها یک ستون در by یا z قرار داشت توان از قرار دادن () صرف نظر کرد.

تمرین: دیتاست Lahman::Batting رو که داده های مربوط به Baseball هست رو در نظر بگیرین. ابتدا

مقایسه زمانی

برای اینکار می توان از تابع microbenchmark از پکیج microbenchmark استفاده نمود. این تابع، توابع مورد نظر و تعداد اجرا را می گیرد.

تمرین: دیتاست iris را به data.table تبدیل کرده و سپس میانگین Sepal.Length را به ازای هر گونه بگویید. همچنین این مقدار را به ازای حروف اول گونه ها بدست آورید.

تمرین: با گروه بندی بر اساس رند شده یک رقم مساحت sepal بگویید در هر گروه چند مشاهده وجود دارد.

Chaining

برای انجام به صورت متوالی کافی است بعد از اولین [] بار دیگر عملیات جدید را بلافاصله بعد از آن در [] جدید بنویسیم .

تمرین: با استفاده از دیتاست iris، میانگین هر کدام از متغیرها را بر اساس گونه بدست آورده و سپس نتیجه را بر اساس گونه به صورت نزولی مرتب کنید.

در مثال قبل فرض کنید ۱۰۰ ستون داشتیم. حالا مجبور به استفاده از روشهای بهتر می بودیم.

SD

SD . به صورت پیش فرض تمام مقادیر تمام ستونها به غیر از ستونهایی که در by آمده اند را در خود نگه می دارد و تنها در بخش ز قابل استفاده است.

استفاده کرده و SDcols. استفاده می کنید برای انتخاب برخی ستونها باید از SD. اگر از ستونهای مورد نظر رو برابر یا اندیشیدن آن قرار دهید

IS

برای حذف، افزودن یا به روز کردن یک ستون استفاده می شود. به این صورت که ستونهای دست چپ برابر مقادیر دست راست قرار می گیرند.

تمرین: یک DT بسازید که یک ستون آن شامل حروف تکراری و ستون دیگر شامل تعدادی عدد باشد. پس با استفاده از اپراتور =: جمع به ازای هر حرف را به آن اضافه نموده، سپس مقادیر ستون دوم را ۵ واحد زیاد کنید.

برای حذف کردن یک یا چند ستون کافی است آنها را برابر NULL قرار دهیم.

اگر چند ستون را به یک متغیر داده و می خواهید با استفاده از متغیر آنها را حذف کنید می بایست نام متغیر داخل پرانتز قرار گیرد. در غیر این صورت ستون یا ستونهای جدید ایجاد دو در جا حذف می شوند.

Outlier یا داده پرت

داده های پرت داده هایی هستند که با سایر مشاهدات اختلاف فاحشی دارند.

دلایل بروز

اتفاقات خیلی خاص مانند وزن بالا یا قد خیلی بلند یک انسان

ایرادات اندازه گیری تجهیزات نا دقیق

خطای محاسباتی

ایراد در ورود داده

کد گذاری ناصحیح ورود برای کسانی که کدشون پر نشده .

باید در موردشون تصمیم گیری بشه که می خوانم -----

تابع hist

تابع boxplot

تابع table

یک جدول از فراوانی مقادیر غیر عددی می سازد.

Plot

کدهای تکمیلی از Rhistory

کد: (۴) summarize / group_by

```
iris |>
  group_by(Species) |>
  summarize(msl = mean(Sepal.Length),
            stdtl = sd(Sepal.Length),
            count = n())
```

خروجی:

```
# A tibble: 3 × 4
  Species      msl stdtl count
  <fct>      <dbl> <dbl> <int>
1 setosa      5.01 0.352   50
2 versicolor  5.94 0.516   50
3 virginica   6.59 0.636   50

# A tibble: 3 × 4
  Species      msl stdtl count
  <fct>      <dbl> <dbl> <int>
1 setosa      5.01 0.352   50
2 versicolor  5.94 0.516   50
```

3 virginica	6.59	0.636	50
-------------	------	-------	----

پیوست: منابع کد

r1.Rhistory	-> مبانی، متغیر، بردار، ماتریس
-------------	--------------------------------

r2.Rhistory	-> factor, data.frame, list, ggplot2
-------------	--------------------------------------

3.Rhistory	-> if / while / for / paste
------------	-----------------------------

3-2.Rhistory	-> function, package, regex, date/time
--------------	--

4.Rhistory	-> import, Excel, DB, dplyr pipe
------------	----------------------------------